

UNIP — UNIVERSIDADE PAULISTA
CIÊNCIA DA COMPUTAÇÃO
7º/8º semestre

Atividades Práticas Supervisionadas (APS)

Tema: “Desenvolvimento de uma aplicação de sistema distribuído para dispositivo móvel”

EcoRadar Urbano: aplicação distribuída para gerenciamento de informações ambientais urbanas

Integrantes do grupo:

Eduardo Zanella Bento - G783590

Arthur Kenji Aizawa Bicas - G801555

Marcelo Marques Pereira - G807BC0

Professor responsável: Fábio

UNIP JK
São José do Rio Preto
2026

ÍNDICE

1 OBJETIVO E MOTIVAÇÃO DO TRABALHO.....	5
1.1 Objetivo geral.....	5
1.2 Objetivos específicos.....	5
1.3 Motivação.....	5
2 INTRODUÇÃO.....	7
2.1 Sistemas distribuídos e computação móvel.....	7
2.2 Cidades inteligentes, sensoriamento e participação cidadã.....	7
2.3 Visão geral da solução.....	8
2.4 Organização do documento.....	8
3 FUNDAMENTOS DAS TECNOLOGIAS PARA DISPOSITIVOS MÓVEIS	
ESCOLHIDAS (CONCEITOS GERAIS).....	10
3.1 Plataformas móveis e abordagens de desenvolvimento.....	10
3.2 React Native e Expo no projeto.....	10
3.3 Recursos do dispositivo utilizados.....	11
3.4 Fundamentos de sistemas distribuídos aplicados.....	12
3.4.1 Transparências, escalabilidade e tolerância a falhas.....	12
3.4.2 Comunicação síncrona, assíncrona e publish/subscribe.....	12
3.4.3 MQTT e AMQP.....	12
3.4.4 WebSocket e Socket.IO.....	13
3.4.5 Microserviços, API Gateway e balanceamento de carga.....	13
3.4.6 Consistência eventual, idempotência, outbox e CQRS.....	14
3.4.7 Circuit breaker, retry e observabilidade.....	14
3.4.8 Containerização.....	15
3.5 Dados geoespaciais com PostGIS.....	15
3.6 Índice de Qualidade do Ar e inversão térmica.....	15
4 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO.....	17
4.1 Elementos e ferramentas.....	17
4.2 Ambiente de desenvolvimento.....	19
4.3 Metodologia.....	20
4.4 Requisitos funcionais.....	20
4.5 Requisitos não funcionais.....	22
4.6 Atores e casos de uso.....	23

4.7 Estratégia de testes.....	24
4.8 Riscos e mitigação.....	25
5 PROJETO (ESTRUTURA E MÓDULOS) DO PROGRAMA.....	27
5.1 Arquitetura geral.....	27
5.2 Microsserviços.....	28
5.3 Comunicação por eventos.....	29
5.4 Fluxos principais.....	29
5.5 Modelo de dados.....	31
5.6 Estrutura do aplicativo e navegação.....	32
5.7 Tratamento de erros e resiliência.....	33
6 RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA.....	34
6.1 Métricas do código.....	34
6.2 Trechos selecionados.....	34
6.2.1 Motor de regras de alertas.....	34
6.2.2 Publicador do outbox.....	35
6.2.3 Idempotência na criação de ocorrências.....	36
6.2.4 Consultas PostGIS por manancial e por raio.....	37
6.2.5 Cálculo do IQAr.....	38
6.2.6 Circuit breaker da Open-Meteo.....	39
6.2.7 Gateway Nginx: balanceamento e WebSocket.....	40
6.2.8 Fila offline do aplicativo.....	41
6.2.9 Componente de mapa por plataforma.....	41
7 APRESENTAÇÃO DO PROGRAMA EM FUNCIONAMENTO.....	43
7.1 Como as evidências foram produzidas.....	43
7.2 Acesso, cadastro e tratamento de erros.....	43
7.3 Mapa ambiental.....	44
7.4 Registro de ocorrência com foto, GPS e manancial.....	45
7.5 Detalhe, confirmação colaborativa e busca por proximidade.....	46
7.6 Qualidade ambiental.....	47
7.7 Tempo real entre dois dispositivos.....	48
7.8 Alerta disparado.....	49
7.9 Modo offline e sincronização.....	50
7.10 Relatórios exportados.....	51
7.11 Status do sistema e painel do agente.....	52

7.12 Tema escuro e outro aparelho.....	53
7.13 Testes de tolerância a falhas.....	54
7.14 Resultados dos testes.....	57
7.15 Teste de carga: uma e duas réplicas.....	57
7.16 Infraestrutura em execução.....	58
7.17 Defeitos encontrados pelos testes e corrigidos.....	59
8 CONSIDERAÇÕES FINAIS: FORMAÇÃO E INTERDISCIPLINARIDADE.....	61
8.1 Interdisciplinaridade.....	61
8.2 Contribuição para a formação.....	62
8.3 Limitações e trabalhos futuros.....	62
9 BIBLIOGRAFIA.....	64
10 FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS.....	68

1 OBJETIVO E MOTIVAÇÃO DO TRABALHO

1.1 Objetivo geral

Desenvolver o **EcoRadar Urbano**, uma aplicação de sistema distribuído para dispositivos móveis que permite a cidadãos, agentes públicos e sensores ambientais registrar, consultar e acompanhar em tempo real informações ambientais urbanas: ocorrências como alagamentos, queimadas, invasão de mananciais, desmatamento e descarte irregular de lixo, e indicadores como o Índice de Qualidade do Ar (IQA_r), o nível dos córregos e a inversão térmica, com uma infraestrutura distribuída que continua funcionando quando parte do sistema falha.

1.2 Objetivos específicos

- a) construir um aplicativo multiplataforma (Android, iOS e web) em React Native com Expo, com mapa, registro de ocorrência com foto e GPS, uso sem conexão e notificações em tempo real;
- b) dividir o backend em seis microsserviços, cada um com o próprio schema de banco de dados, atrás de um API Gateway;
- c) integrar os serviços por mensageria assíncrona (AMQP), receber a telemetria dos sensores por MQTT e entregar os eventos aos aparelhos por WebSocket (Socket.IO);
- d) armazenar e consultar dados geoespaciais com PostgreSQL e PostGIS, identificando automaticamente ocorrências dentro de áreas de manancial;
- e) calcular o IQA_r com as faixas divulgadas pela CETESB e detectar inversão térmica a partir das leituras;
- f) aplicar padrões de tolerância a falhas e consistência (balanceamento, *outbox*, idempotência, CQRS, *circuit breaker* e *retry*) e comprová-los com testes de falha controlada;
- g) gerar relatórios com indicadores e exportação em PDF e CSV;
- h) registrar evidências reais com testes unitários, de integração, ponta a ponta, distribuídos e de carga.

1.3 Motivação

As grandes cidades brasileiras convivem com problemas ambientais que se somam. No inverno, a inversão térmica dificulta a dispersão dos poluentes e agrava

episódios de má qualidade do ar; em dias de chuva forte, córregos canalizados transbordam e alagam vias; nas bordas da mancha urbana, a ocupação irregular e o desmatamento avançam sobre as áreas que protegem os reservatórios de abastecimento; queimadas e descarte irregular de lixo completam o quadro, junto com uma mobilidade apoiada no transporte rodoviário, que também é fonte de emissões. A cidade de São Paulo foi adotada como referência do protótipo: as represas Guarapiranga e Billings são protegidas por leis estaduais específicas (SÃO PAULO, 2006; 2009) e a CETESB publica o IQAr das estações da sua rede de monitoramento (CETESB, 2026).

Muitos desses problemas são percebidos primeiro por quem está no local. Um cidadão com um celular consegue registrar, em segundos, uma foto georreferenciada de um córrego transbordando ou de uma construção irregular na margem de uma represa — é a ideia de “cidadãos como sensores” descrita por Goodchild (2007). Essa informação só tem valor se chegar depressa a quem pode agir e for confirmada e cruzada com dados de sensores. A Lei nº 14.850/2024, que instituiu a Política Nacional de Qualidade do Ar, e a Resolução CONAMA nº 506/2024 reforçam essa necessidade ao prever a divulgação do IQAr à população em tempo real, horário ou diário (BRASIL, 2024a; 2024b).

Esse cenário justifica uma arquitetura distribuída e orientada a eventos. Os dados chegam de muitas fontes ao mesmo tempo (aparelhos, estações e uma API externa), o volume varia com os eventos climáticos, falhas parciais são inevitáveis (rede móvel instável, um serviço reiniciando, a API externa fora do ar) e um alerta perde valor se chega atrasado. Por isso o grupo optou por serviços independentes, mensageria com garantia de entrega, réplicas atrás de um balanceador e envio ativo de notificações, em vez de um sistema único consultado periodicamente.

2 INTRODUÇÃO

2.1 Sistemas distribuídos e computação móvel

Para Tanenbaum e Van Steen (2007), um sistema distribuído é um conjunto de computadores independentes que se apresenta aos usuários como um sistema único e coerente. Coulouris et al. (2013) destacam que seus componentes, localizados em computadores ligados em rede, comunicam-se e coordenam suas ações apenas pela troca de mensagens, o que traz três consequências de projeto: concorrência entre os componentes, ausência de um relógio global e falhas independentes — uma parte pode parar enquanto as demais continuam. Essas três características aparecem o tempo todo no EcoRadar Urbano: dois aparelhos registram ocorrências ao mesmo tempo, eventos chegam fora de ordem e o broker de mensagens pode cair sem que o restante do sistema pare.

A computação móvel acrescenta um participante com características próprias. O *smartphone* é ao mesmo tempo cliente e fonte de dados: tem GPS, câmera e armazenamento local, mas opera com conectividade intermitente, bateria limitada e redes de latência variável. Em vez de tratar a falta de rede como exceção, o aplicativo precisa ser projetado para funcionar desconectado (abordagem *offline-first*), guardar o que o usuário fez e sincronizar depois sem gerar duplicidade. Do lado do servidor, isso exige operações idempotentes, porque o mesmo pedido pode chegar mais de uma vez.

2.2 Cidades inteligentes, sensoriamento e participação cidadã

O conceito de cidade inteligente reúne o uso de tecnologias da informação para melhorar serviços urbanos e a qualidade de vida, com indicadores padronizados, por exemplo, pela ABNT NBR ISO 37122 (ABNT, 2020). Parte desses dados vem de sensores espalhados pela cidade — estações de qualidade do ar, pluviômetros, medidores de nível de córregos e perfis de temperatura —, dispositivos de Internet das Coisas (IoT) que publicam leituras periódicas usando protocolos leves como o MQTT (OASIS, 2014).

A outra parte vem das pessoas. O *crowdsourcing* de dados urbanos, chamado por Goodchild (2007) de informação geográfica voluntária, transforma moradores em observadores distribuídos pelo território. Essa fonte é rápida e capilar, mas heterogênea: um relato pode estar incompleto, repetido ou errado. Por isso o

EcoRadar combina as duas fontes e acrescenta mecanismos de validação — confirmação colaborativa por outros cidadãos, triagem por agentes com histórico de status e regras automáticas que cruzam relatos próximos no tempo e no espaço.

2.3 Visão geral da solução

O EcoRadar Urbano é formado por um aplicativo móvel e por um backend distribuído executado em contêineres Docker. O aplicativo, escrito em TypeScript com React Native e Expo, foi feito para o Android e o iOS, pelo Expo Go, e também roda no navegador. Toda a comunicação passa por um API Gateway Nginx, que distribui as requisições entre seis microsserviços Node.js: autenticação, ocorrências (com duas réplicas), dados ambientais, alertas, relatórios e um simulador de sensores. Os dados ficam em PostgreSQL com a extensão PostGIS, com um schema isolado por serviço, e a comunicação assíncrona usa o RabbitMQ, que atende tanto AMQP (eventos entre serviços) quanto MQTT (leituras dos sensores).

Três fluxos resumem o funcionamento. Quando um cidadão registra uma ocorrência, ela é gravada junto com um evento na mesma transação e publicada no broker; o serviço de alertas avalia as regras e todos os aparelhos conectados recebem a novidade por WebSocket. Enquanto isso, seis estações virtuais publicam leituras a cada cinco segundos, o serviço ambiental calcula o IQAr, detecta inversão térmica e nível crítico de córregos e gera eventos que também podem virar alertas. Por fim, dados reais de qualidade do ar e clima são obtidos da API pública Open-Meteo, protegida por *circuit breaker* e cache. Relatórios são calculados a partir de uma visão de leitura própria (CQRS) e exportados em PDF ou CSV.

O sistema foi executado e testado de ponta a ponta: 105 testes unitários, 20 testes de integração pelo gateway, 16 cenários ponta a ponta com Playwright em *viewport* de celular, cinco testes de falha controlada e um teste de carga comparando uma e duas réplicas. Todas as evidências (prints, vídeos, logs e relatórios) estão na pasta registros/ do repositório e são apresentadas na seção 7.

2.4 Organização do documento

A seção 1 apresentou o objetivo e a motivação do trabalho. A seção 3 reúne os fundamentos das tecnologias móveis e dos conceitos de sistemas distribuídos usados, sempre ligados ao uso que tiveram no projeto. A seção 4 descreve o plano de desenvolvimento: ferramentas e justificativas, ambiente, metodologia, requisitos,

casos de uso, estratégia de testes e riscos. A seção 5 detalha a estrutura do programa — arquitetura, microsserviços, eventos, fluxos, modelo de dados, organização do aplicativo e tratamento de erros. A seção 6 traz as métricas do código e trechos comentados dos pontos centrais. A seção 7 mostra o programa em funcionamento com os prints das execuções reais, incluindo os testes de tolerância a falhas e de carga. A seção 8 discute a interdisciplinaridade, a contribuição do trabalho para a formação do grupo e as limitações. Seguem a bibliografia e a ficha de atividades práticas supervisionadas.

3 FUNDAMENTOS DAS TECNOLOGIAS PARA DISPOSITIVOS MÓVEIS ESCOLHIDAS (CONCEITOS GERAIS)

3.1 Plataformas móveis e abordagens de desenvolvimento

O mercado de *smartphones* é dominado por duas plataformas: o Android, mantido pelo Google sobre um núcleo Linux, com aplicativos escritos principalmente em Kotlin ou Java, e o iOS, da Apple, com aplicativos em Swift ou Objective-C. Cada uma tem seu SDK, suas lojas, suas regras de permissão e seus componentes visuais. Há três caminhos para atender às duas:

- a) **desenvolvimento nativo**: um projeto por plataforma, com acesso total às APIs e o melhor desempenho, ao custo de manter dois códigos e duas equipes de conhecimento;
- b) **desenvolvimento híbrido**: uma aplicação web empacotada dentro de uma *WebView* (Cordova, Ionic/Capacitor), com um só código, mas interface web e acesso a recursos do aparelho por *plugins*;
- c) **desenvolvimento multiplataforma com interface nativa**: um só código que gera componentes nativos (React Native) ou desenha a interface com motor próprio (Flutter).

O grupo escolheu o React Native com Expo. O React Native usa JavaScript/TypeScript e React, e cada componente declarado (View, Text, Image) vira um componente nativo real da plataforma (REACT NATIVE, 2026a). Pesaram na decisão: a mesma linguagem do backend (TypeScript); o Expo Go, que executa o aplicativo no celular a partir de um QR code, sem compilar projeto Android ou iOS; e o suporte à web (react-native-web), que tornou possível testar o aplicativo automaticamente em *viewport* de celular com o Playwright no computador de desenvolvimento, que não tinha Android SDK.

3.2 React Native e Expo no projeto

O Expo (SDK 57, com React Native 0.86.3 e React 19.2.3) é um conjunto de ferramentas e módulos em torno do React Native (EXPO, 2026a). O **Metro** empacota o código JavaScript e o serve pela rede local ao **Expo Go**, aplicativo instalado no celular que carrega esse pacote e expõe os módulos nativos do SDK. O **Expo Router** implementa rotas baseadas em arquivos (EXPO, 2026b): cada arquivo em *mobile/src/app* é uma tela, grupos entre parênteses, como (abas), definem a

barra de abas, e nomes entre colchetes, como `ocorrencia/[id].tsx`, são rotas dinâmicas.

Quando uma funcionalidade precisa de implementações diferentes por plataforma, o Metro escolhe o arquivo pela extensão: `Mapa.native.tsx` no Android e no iOS, `Mapa.web.tsx` no navegador (REACT NATIVE, 2026b). O projeto usa esse recurso no mapa (react-native-maps com Google Maps no Android e Apple Maps no iOS; Leaflet com OpenStreetMap na web), no armazenamento, nas notificações, na mídia e na exportação de arquivos. O estado local fica no Zustand (sessão, preferências, conexão e fila offline) e as consultas à API passam pelo TanStack Query, cujo cache é persistido no aparelho por 24 horas em modo *offline-first*; o Axios faz as requisições com tempo limite de 10 segundos e até três tentativas em falhas transitórias.

3.3 Recursos do dispositivo utilizados

- a) **GPS e geolocalização** (expo-location): o aplicativo pede permissão de uso em primeiro plano, obtém a posição com precisão balanceada e, se ela não vier a tempo, usa a última posição conhecida; o usuário ainda pode ajustar o ponto arrastando um pino no mapa;
- b) **câmera e galeria** (expo-image-picker): a foto da ocorrência é capturada com qualidade 0,6 e sem metadados EXIF, o que reduz o envio e protege a privacidade;
- c) **armazenamento local e seguro**: o AsyncStorage guarda a fila offline, o cache de consultas e as preferências; o token de acesso fica no expo-secure-store, que usa o Keychain do iOS e o Keystore do Android;
- d) **notificações** (expo-notifications): cada alerta recebido em tempo real gera uma notificação local e um *badge* na aba Alertas, respeitando o raio configurado pelo usuário;
- e) **detecção de conectividade** (NetInfo): além de saber se há rede, o aplicativo testa se o gateway responde em `/health`; ao recuperar a conexão, a fila de envios pendentes é sincronizada automaticamente.

3.4 Fundamentos de sistemas distribuídos aplicados

3.4.1 *Transparências, escalabilidade e tolerância a falhas*

Tanenbaum e Van Steen (2007) descrevem formas de transparência que escondem do usuário a natureza distribuída do sistema. No EcoRadar, o aplicativo conhece um único endereço, o do gateway (transparência de localização); não sabe que o serviço de ocorrências tem duas réplicas (transparência de replicação); e falhas transitórias são mascaradas por novas tentativas no gateway e no cliente (transparência de falha, parcial). A escalabilidade horizontal vem de serviços sem estado em memória — todo estado fica no banco ou no broker —, de modo que basta subir mais réplicas atrás do balanceador. A tolerância a falhas combina redundância (réplicas), detecção (*health checks*), isolamento (um serviço fora não derruba os outros) e recuperação automática (reconexão e reenvio).

3.4.2 *Comunicação síncrona, assíncrona e publish/subscribe*

Na comunicação síncrona, o cliente envia uma requisição e aguarda a resposta. É o modelo do HTTP e das APIs REST, em que recursos são identificados por URLs, manipulados por métodos como GET, POST e PATCH e respondidos com códigos de status padronizados (FIELDING; NOTTINGHAM; RESCHKE, 2022). O aplicativo usa esse modelo para tudo o que precisa de resposta imediata. Já a comunicação assíncrona desacopla emissor e receptor no tempo: a mensagem é entregue a um intermediário (*broker*), que a guarda até que o consumidor possa processá-la. No modelo *publish/subscribe*, quem publica não conhece quem consome; cada interessado assina os tipos de evento que deseja. Assim, um único evento ocorrência.criada chega ao serviço de alertas e ao de relatórios.

3.4.3 *MQTT e AMQP*

O **MQTT** é um protocolo *publish/subscribe* leve sobre TCP, pensado para dispositivos com poucos recursos (OASIS, 2014). As mensagens são publicadas em tópicos hierárquicos, e as assinaturas aceitam curingas (+ para um nível, # para vários). O nível de qualidade de serviço define a garantia de entrega: QoS 0 (no máximo uma vez), QoS 1 (pelo menos uma vez) e QoS 2 (exatamente uma vez). No projeto, cada estação publica em `ecoradar/sensores/{id}/leituras` com QoS 1, e o serviço ambiental assina `ecoradar/sensores/+leituras`.

O **AMQP 0-9-1**, modelo usado pelo RabbitMQ, separa o roteamento da armazenagem (RABBITMQ, 2026a): o produtor publica em uma *exchange* com uma chave de roteamento; filas se ligam à *exchange* por *bindings*; e os consumidores leem das filas, confirmando cada mensagem (*ack*). Uma *exchange* do tipo *topic* aceita padrões nas ligações (* para uma palavra, # para várias). O projeto usa a *exchange* *ecoradar.events* com filas duráveis por serviço, mensagens persistentes, *publisher confirms* — o produtor só considera a mensagem publicada após a confirmação do broker (RABBITMQ, 2026c) — e *dead letter exchange* para mensagens que falham repetidamente (RABBITMQ, 2026d). Como o RabbitMQ implementa os dois (o MQTT por meio de um *plugin*), um único broker atende às duas necessidades (RABBITMQ, 2026b).

3.4.4 WebSocket e Socket.IO

No HTTP, o servidor só responde quando é chamado. O **WebSocket** (FETTE; MELNIKOV, 2011) abre, a partir de um *handshake* HTTP com o cabeçalho Upgrade, uma conexão persistente e bidirecional pela qual o servidor pode enviar dados a qualquer momento, sem que o cliente precise consultar periodicamente (*polling*). O **Socket.IO** acrescenta sobre ele eventos nomeados, reconexão automática com *backoff*, salas e *middlewares* de autenticação (SOCKET.IO, 2026). No EcoRadar, o aplicativo abre a conexão autenticada com o token JWT e recebe os eventos *ocorrencia:nova*, *ocorrencia:atualizada*, *alerta:novo* e *alerta:encerrado*; o Nginx repassa a conexão com os cabeçalhos de *upgrade* (NGINX, 2026b).

3.4.5 Microsserviços, API Gateway e balanceamento de carga

Microsserviços são serviços pequenos e independentes, modelados em torno de uma capacidade de negócio, implantados separadamente e donos dos próprios dados (NEWMAN, 2022). Os ganhos — implantação e escala independentes, isolamento de falhas, liberdade tecnológica por serviço — têm como custo a complexidade operacional e a necessidade de lidar com consistência entre bancos distintos. O padrão *database per service* (RICHARDSON, 2018) foi seguido à risca: cada serviço tem um schema e um usuário de banco próprios e nenhum lê as tabelas de outro. Um **API Gateway** é o ponto único de entrada que roteia as requisições para o serviço certo e concentra funções transversais (RICHARDSON, 2026b). O **balanceamento de carga** distribui as requisições entre réplicas de um

serviço; o Nginx usa por padrão o algoritmo *round-robin*, marca temporariamente como indisponível a réplica que falha e pode repetir a requisição na próxima (NGINX, 2026a).

3.4.6 Consistência eventual, idempotência, outbox e CQRS

Sem transações distribuídas, os dados de serviços diferentes convergem com algum atraso: é a **consistência eventual** (KLEPPMANN, 2017). No teste de integração que consulta as estatísticas logo após registrar uma ocorrência e mudar seu status, foi preciso aguardar 1,06 s até a visão de leitura do serviço de relatórios refletir as duas alterações. Como as mensagens são entregues “pelo menos uma vez”, o mesmo evento pode chegar duas vezes, e as operações precisam ser **idempotentes** — repeti-las tem o mesmo efeito que executá-las uma vez. O aplicativo gera uma chave UUID v4 para cada registro e o servidor recusa duplicatas por uma restrição de unicidade; os consumidores guardam o identificador de cada evento já processado (tabela de *inbox*).

Outro problema clássico é a escrita dupla: gravar no banco e publicar no broker não é atômico, e uma falha entre os dois passos perde o evento ou publica algo que não foi salvo. O padrão **Transactional Outbox** resolve isso gravando o evento numa tabela outbox na mesma transação local da alteração; um publicador separado envia os eventos pendentes ao broker e os marca como publicados (RICHARDSON, 2026a). Já o **CQRS** separa o modelo de escrita do modelo de leitura (FOWLER, 2011): o serviço de relatórios mantém uma visão própria, alimentada por eventos e otimizada para consultas agregadas, sem consultar o banco do serviço de ocorrências.

3.4.7 Circuit breaker, retry e observabilidade

Chamar um serviço externo lento ou fora do ar pode prender recursos e propagar a falha. O **circuit breaker** (NYGARD, 2018; FOWLER, 2014) monitora as chamadas: com muitas falhas, o circuito **abre** e as chamadas seguintes falham imediatamente; após um intervalo, fica **meio-aberto** e deixa passar uma chamada de teste; se ela der certo, **fecha** de novo. Complementam o padrão o *timeout* por chamada, o *retry* com *backoff* exponencial e *jitter* (esperas crescentes com variação aleatória) e a degradação graciosa, que entrega o último dado válido em vez de um erro. Para operar tudo isso é preciso **observabilidade**: logs estruturados em JSON,

um identificador de correlação (X-Request-Id) que acompanha a requisição do gateway aos serviços e aos consumidores de eventos, e verificações de saúde separadas em *liveness* (/health, o processo está no ar) e *readiness* (/ready, banco e broker acessíveis).

3.4.8 Containerização

Um contêiner empacota a aplicação com suas dependências numa imagem imutável e a executa isolada do sistema hospedeiro, com custo muito menor que o de uma máquina virtual. O **Docker Compose** descreve, num único arquivo, todos os contêineres da aplicação, suas redes, volumes, variáveis de ambiente e verificações de saúde (DOCKER, 2026). No projeto, o `docker-compose.yml` sobe dez contêineres, e cada serviço só inicia depois que o banco e o broker estão saudáveis (`depends_on` com `service_healthy`).

3.5 Dados geoespaciais com PostGIS

O PostGIS acrescenta ao PostgreSQL tipos, funções e índices espaciais (POSTGIS, 2026). Toda coordenada precisa de um sistema de referência, identificado por um SRID; o GPS dos celulares usa o WGS 84, de SRID 4326, com latitude e longitude em graus. As ocorrências são gravadas como `geometry(Point, 4326)` e os mananciais como `geometry(MultiPolygon, 4326)`, ambos com índices GiST. Duas consultas são centrais: a **busca por raio**, com `ST_DWithin` sobre o tipo `geography` (distância em metros, usando o índice) e `ST_Distance` para ordenar pela proximidade; e o **ponto-em-polígono**, com `ST_Contains`, que indica se uma ocorrência está dentro da área de proteção da Guarapiranga ou da Billings (polígonos aproximados, para fins didáticos).

3.6 Índice de Qualidade do Ar e inversão térmica

O IQAr é uma ferramenta de comunicação que traduz a concentração de cada poluente numa escala única associada a efeitos sobre a saúde. O projeto usa as faixas publicadas pela CETESB (Tabela 1), que remetem à Resolução CONAMA nº 491/2018 (CETESB, 2026; BRASIL, 2018). Para cada poluente, o índice é obtido por interpolação linear dentro da faixa em que a concentração se encontra, e o IQAr divulgado é o maior entre os poluentes:

$$\text{IQAr} = \text{lini} + (\text{lfin} - \text{lini}) / (\text{Cfin} - \text{Cini}) \times (\text{C} - \text{Cini})$$

em que l_{ini} e l_{fin} são os índices inicial e final da faixa, C_{ini} e C_{fin} as concentrações inicial e final e C a concentração medida. A Resolução CONAMA nº 506/2024 revogou a maior parte da nº 491/2018 e passou a definir o cálculo do IQAr no seu Anexo II, que mantém a mesma equação e a mesma faixa N1 (BRASIL, 2024b).

[CONFERIR: comparar as faixas N2 a N5 da tabela com o Anexo II da Resolução CONAMA nº 506/2024.] Duas simplificações foram documentadas no código: a metodologia oficial usa médias de 24 h, 8 h ou 1 h, enquanto o protótipo aplica as faixas à média móvel das últimas 12 leituras (cerca de um minuto), para que a demonstração reaja em tempo real; e, como a faixa N5 não tem limite superior, adotou-se o índice 400 nas concentrações de referência de emergência, com teto de 500.

Tabela 1 – Faixas do IQAr usadas no projeto (concentrações em $\mu\text{g}/\text{m}^3$; CO em ppm)

Poluente (média)	N1 Boa	N2 Moderada	N3 Ruim	N4 Muito Ruim	N5 Péssima
Índice	0–40	41–80	81–120	121–200	>200
MP10 (24 h)	0–45	>45–100	>100–150	>150–250	>250
MP2,5 (24 h)	0–15	>15–50	>50–75	>75–125	>125
O3 (8 h)	0–100	>100–130	>130–160	>160–200	>200
CO (8 h)	0–9	>9–11	>11–13	>13–15	>15
NO2 (1 h)	0–200	>200–240	>240–320	>320– 1130	>1130
SO2 (24 h)	0–40	>40–50	>50–125	>125–800	>800

Fonte: elaborado pelos autores (2026), com base em CETESB (2026).

A **inversão térmica** ocorre quando uma camada de ar mais quente se forma acima de uma camada mais fria junto ao solo. Normalmente a temperatura diminui com a altitude e o ar aquecido na superfície sobe, levando os poluentes; na inversão, o ar frio fica preso embaixo e os poluentes se acumulam. No EcoRadar, as estações com sensor de perfil térmico informam a temperatura na superfície e a 300 m: há inversão quando a temperatura a 300 m é maior que a da superfície, classificada como fraca (diferença menor que 1 °C), moderada (a partir de 1 °C) ou forte (a partir de 3 °C), e o alerta correspondente recebe severidade alta quando a diferença chega a 3 °C.

4 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO

Esta seção descreve os elementos e ferramentas usados, o ambiente em que o sistema foi construído, a forma de trabalho, os requisitos, os atores e casos de uso, a estratégia de testes e os riscos considerados.

4.1 Elementos e ferramentas

A Tabela 2 resume a pilha tecnológica. O critério principal foi usar uma única linguagem (TypeScript) do aplicativo ao backend, ferramentas de código aberto que rodassem inteiramente num computador pessoal e tecnologias que expusessem, de forma visível, os conceitos de sistemas distribuídos estudados.

Tabela 2 – Pilha tecnológica, justificativas e alternativas descartadas

Camada	Tecnologia	Por que foi escolhida	Alternativas descartadas
App	React Native 0.86 + Expo SDK 57, TypeScript	Um código para Android, iOS e web; Expo Go dispensa build nativo; mesma linguagem do backend	Kotlin/Swift nativos (dois códigos); Flutter (Dart); Ionic (WebView)
App	Expo Router	Rotas por arquivos, abas e rotas dinâmicas sem configuração manual	React Navigation configurado à mão
App	React Native Paper (Material 3)	Componentes acessíveis prontos e tema claro/escuro	Estilização manual completa
App	Zustand + TanStack Query + Axios	Estado simples persistido; cache de consultas offline; interceptadores de erro e retry	Redux Toolkit (mais verboso)
App	react-native-maps / Leaflet + OpenStreetMap	Mapas nativos no celular e mapa aberto na web, sem chave paga	Mapbox e Google Maps JS (exigem chave)

Camada	Tecnologia	Por que foi escolhida	Alternativas descartadas
Backend	Node.js 24 LTS + Fastify 5	Alto desempenho, validação por esquema, hooks e Swagger integrados	Express (sem esquemas nativos); NestJS (mais cerimônia); Spring Boot (outra linguagem)
Backend	Zod 4	Um só esquema valida a entrada e gera a documentação OpenAPI	Joi, validação manual
Dados	PostgreSQL 18 + PostGIS 3.6	Transações ACID para o outbox e consultas espaciais com índice	MongoDB (sem transação com outbox tão simples); MySQL Spatial
Dados	Drizzle ORM + drizzle-kit	Consultas tipadas próximas do SQL e migrações versionadas	Prisma (suporte limitado a PostGIS); TypeORM
Mensageria	RabbitMQ 4.3 (AMQP + plugin MQTT)	Um só broker para eventos entre serviços e telemetria de sensores	Kafka (mais pesado, sem MQTT); Mosquitto + outro broker
Tempo real	Socket.IO 4.8	Reconexão automática, eventos nomeados, autenticação no handshake	WebSocket puro; SSE (unidirecional); Firebase (serviço externo)
Gateway	Nginx 1.30	Proxy reverso maduro, round-robin, WebSocket e logs em JSON	Traefik; Kong; gateway escrito em Node
Resiliência	opossum 10, backoff próprio	Circuit breaker testado; retry com jitter reaproveitado em todos os serviços	Implementação própria do breaker

Camada	Tecnologia	Por que foi escolhida	Alternativas descartadas
Relatórios	pdfkit + CSV	Geração de PDF no servidor, sem navegador	Puppeteer (pesado)
Infra	Docker + Docker Compose	Sobe tudo com um comando, com healthchecks e ordem de inicialização	Kubernetes (excessivo para o escopo)
Testes	Vitest, Playwright, autocannon	Unitários e integração rápidos; E2E com prints e vídeos; carga via gateway	Jest; Cypress/Detox; k6

Fonte: elaborado pelos autores (2026), com versões de registros/ambiente/versoes.txt.

4.2 Ambiente de desenvolvimento

O sistema foi desenvolvido e executado num único computador com Windows 11 Home (build 26200), processador AMD Ryzen 5 7600 (6 núcleos e 12 *threads*) e 30,9 GB de memória, com o Docker Desktop sobre o WSL 2. As versões efetivamente usadas foram registradas automaticamente pelos scripts do projeto e estão na Tabela 3. O TypeScript foi fixado na versão 6.0.3, e não na 7, porque o typescript-eslint suporta versões anteriores à 6.1 e o modelo do Expo SDK 57 também usa a 6.0.3. Os navegadores do Playwright foram instalados dentro da pasta tests/, sem instalação global.

Tabela 3 – Versões das ferramentas e servidores usados

Ferramenta ou servidor	Versão	Ferramenta ou pacote	Versão
Node.js (máquina)	24.19.0	TypeScript	6.0.3
Node.js (contêineres)	24.21.0	Fastify	5.12.5
npm	12.0.2	Zod	4.6.5
Git	2.55.0	Drizzle ORM	0.45.3
Docker / Compose	29.8.0 / 5.5.1	amqplib / mqtt	2.0.1 / 5.16.0
PostgreSQL	18.6	Socket.IO	4.8.4
PostGIS	3.6.4	opossum	10.0.0

Ferramenta ou servidor	Versão	Ferramenta ou pacote	Versão
RabbitMQ	4.3.6	Expo / React Native	57.0.25 / 0.86.3
Nginx	1.30.5	Playwright / Vitest	1.63.0 / 5.0.2

Fonte: elaborado pelos autores (2026), a partir de registros/ambiente/versoes.txt.

4.3 Metodologia

O desenvolvimento foi incremental, em dez fases registradas no arquivo PROGRESSO.md: ambiente e estrutura; infraestrutura (Compose, PostGIS, RabbitMQ e Nginx); pacote compartilhado e serviço de autenticação; serviço de ocorrências; simulador e serviço ambiental; alertas e relatórios; aplicativo móvel; dados de demonstração, scripts e documentação OpenAPI; testes e evidências; e documentação final. Cada fase entregava uma fatia vertical funcionando e só era encerrada após uma verificação — teste automatizado ou chamada real pelo gateway. O PROGRESSO.md também serviu de diário de decisões, e os *commits* seguiram o padrão *Conventional Commits* (feat, fix, test, docs, chore).

Portões de qualidade foram aplicados a todo o código: checagem de tipos (tsc --noEmit) no backend, no aplicativo e nos testes, ESLint e expo lint sem erros e expo-doctor com as 21 verificações aprovadas. Os próprios testes encontraram dez defeitos, todos corrigidos e registrados — entre eles, a autorização executada depois da validação (o cidadão recebia 422 em vez de 403), o escalonamento de alertas suprimido pela deduplicação, erros de *socket* sob carga causados pelo limite de requisições por conexão *keep-alive* do Nginx e o limite de tentativas de login que contava apenas o IP, bloqueando todos os usuários de uma mesma rede. A lista completa está na seção 7.

4.4 Requisitos funcionais

A Tabela 4 lista os requisitos funcionais implementados e onde cada um é atendido.

Tabela 4 – Requisitos funcionais

ID	Requisito	Onde é atendido
RF01	Cadastrar cidadão e autenticar por e-mail e senha, com sessão de 12 h	auth-service; telas login e cadastro

ID	Requisito	Onde é atendido
RF02	Controlar o acesso por perfil: cidadão, agente e administrador	todos os serviços (JWT)
RF03	Registrar ocorrência com categoria (10), severidade (4), descrição, foto opcional e localização por GPS ajustável no mapa	ocorrencias-service; tela ocorrencia/nova
RF04	Identificar e avisar quando o ponto está em área de manancial	ocorrencias-service (PostGIS)
RF05	Exibir mapa com ocorrências, estações com IQAr e polígonos dos mananciais, com filtro por categoria	aba Mapa
RF06	Listar ocorrências com busca, filtros, cinco ordenações e “perto de mim” por raio	aba Ocorrências
RF07	Exibir detalhe da ocorrência com foto, mapa e histórico de status	tela ocorrencia/[id]
RF08	Permitir confirmação colaborativa (uma por usuário, nunca a própria, só em ocorrências abertas)	ocorrencias-service
RF09	Permitir ao agente alterar o status com comentário obrigatório e transições válidas	tela painel; ocorrencias-service
RF10	Receber leituras de seis estações por MQTT e calcular IQAr, inversão térmica e nível de córregos, com série de 24 h	ambiental-service; aba Ambiente
RF11	Integrar dados de ar e clima da Open-Meteo, indicando se são ao vivo ou de cache	ambiental-service
RF12	Gerar alertas por cinco regras, com deduplicação de 30 min e escalonamento	alertas-service
RF13	Notificar em tempo real com banner, badge e notificação local, conforme o raio escolhido	alertas-service (Socket.IO); app
RF14	Encerrar alertas (agente/admin) e expirá-los após 12 h	alertas-service
RF15	Registrar sem conexão em fila persistente e sincronizar sem duplicar	app (fila offline); tela envios

ID	Requisito	Onde é atendido
RF16	Gerar relatórios com indicadores, gráficos e áreas críticas, exportando PDF e CSV com filtros	relatorios-service; tela relatorios
RF17	Mostrar o status do sistema: saúde e latência dos serviços, réplica, balanceamento, circuit breaker, WebSocket, MQTT, AMQP e outbox	tela status
RF18	Permitir ao administrador disparar cenários do simulador e simular falha da Open-Meteo	sensor-simulator; ambiental-service; painel
RF19	Configurar tema claro/escuro/sistema e raio de alertas (2, 5, 10 ou 20 km)	aba Perfil
RF20	Permitir ao administrador listar usuários e alterar perfis	auth-service

Fonte: elaborado pelos autores (2026).

4.5 Requisitos não funcionais

A Tabela 5 apresenta os requisitos não funcionais e a evidência de que foram atendidos.

Tabela 5 – Requisitos não funcionais

ID	Requisito	Como foi atendido e verificado
RNF0 1	Tolerância a falha de instância	2 réplicas do serviço de ocorrências; com uma parada, 20 de 20 requisições atendidas
RNF0 2	Escalabilidade horizontal	Serviços sem estado atrás do balanceador; +69,5% de vazão com 2 réplicas
RNF0 3	Nenhum evento perdido	Outbox transacional e publisher confirms; com o broker fora por ~20 s, os 5 eventos pendentes foram publicados na volta
RNF0 4	Idempotência	Chave UUID do app com restrição única; inbox nos consumidores; reenvio devolve o original (200)
RNF0 5	Desempenho interativo	Listagem pelo gateway com p95 de 41 ms (1 réplica) e 35 ms (2 réplicas) sob 50 conexões

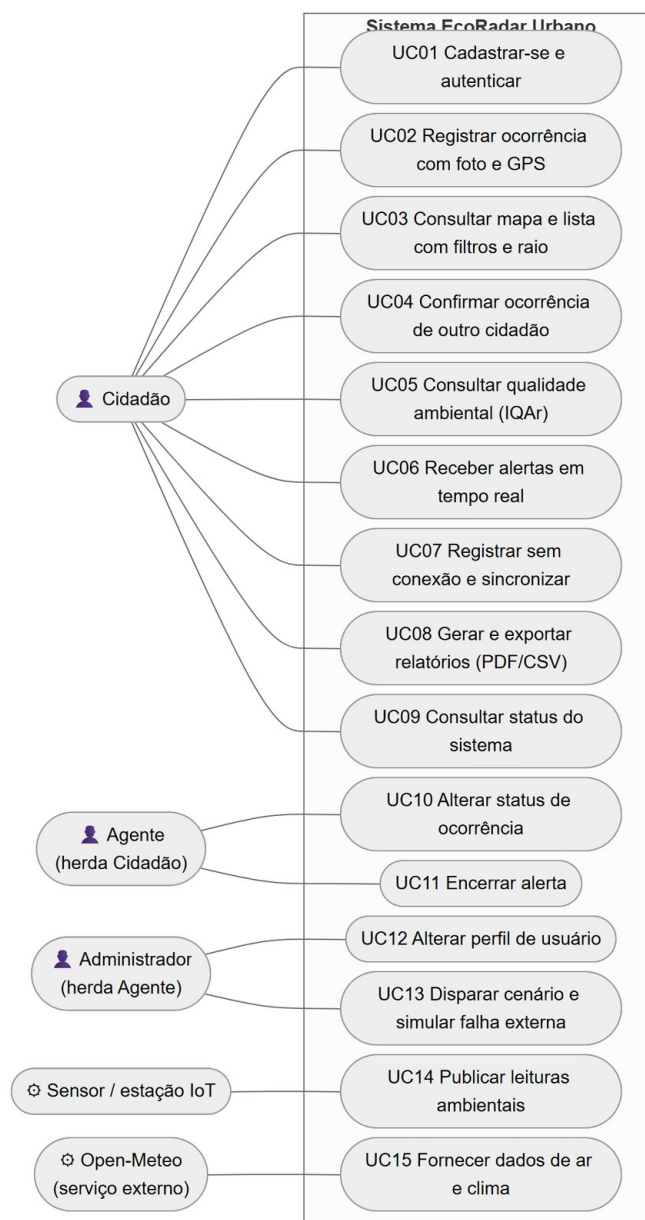
ID	Requisito	Como foi atendido e verificado
RNF0 6	Segurança	bcrypt, JWT de 12 h, limite de login por IP + e-mail, validação Zod, tipo de foto pelos bytes, token no Keychain/Keystore, usuário de banco por serviço, só 2 portas expostas
RNF0 7	Observabilidade	X-Request-Id ponta a ponta, logs JSON, /health e /ready, X-Instance-Id, tela de status
RNF0 8	Portabilidade	Mesmo código para Android, iOS e web; backend em contêineres
RNF0 9	Operação sem conexão	Cache de consultas persistido por 24 h e fila offline com sincronização automática
RNF1 0	Resiliência a serviço externo	Timeout de 5 s, circuit breaker (50% de erro, 30 s) e cache de 10 min persistido
RNF1 1	Usabilidade e acessibilidade	Tema escuro, rótulos de acessibilidade, mensagens em português, paleta de gráficos validada para daltonismo
RNF1 2	Manutenibilidade	TypeScript em todo o código, lint e tipos sem erros, cobertura de 97,47% das linhas das regras de negócio

Fonte: elaborado pelos autores (2026)

4.6 Atores e casos de uso

Cinco atores interagem com o sistema (Figura 1). O **cidadão** registra e consulta ocorrências, confirma relatos de outras pessoas, acompanha a qualidade ambiental, recebe alertas e gera relatórios. O **agente** tem tudo o que o cidadão tem e, além disso, faz a triagem: altera o status das ocorrências e encerra alertas. O **administrador** herda as permissões do agente, altera perfis de usuários e dispara cenários de demonstração e a falha simulada da API externa. Os atores não humanos são o **sensor**, representado pelas estações virtuais que publicam leituras por MQTT, e o **serviço externo** Open-Meteo, que fornece dados reais de qualidade do ar e clima.

Figura 1 – Diagrama de casos de uso



Fonte: elaborado pelos autores (2026).

4.7 Estratégia de testes

Os testes foram organizados em camadas, das mais rápidas e isoladas às que exercitam o sistema inteiro (Tabela 6). Um único comando (scripts/rodar-todos-testes) executa todas elas em sequência e grava as saídas em registros; outro (scripts/coletar-evidencias) sobe a stack, roda os testes e gera a galeria de evidências, de modo que os números deste documento podem ser reproduzidos.

Tabela 6 – Estratégia de testes

Nível	Ferramenta	O que verifica
Unitário	Vitest + cobertura v8	Regras de negócio isoladas: IQAr, inversão, córregos, motor de regras, validações, idempotência, transições de status, outbox com broker simulado, backoff, cache, circuit breaker e relatórios
Integração	Vitest + fetch + Socket.IO	Fluxo completo pela stack real via gateway e respostas de erro 401, 403, 404, 409, 413, 415, 422 e 429
Ponta a ponta	Playwright (Pixel 7 e iPhone 14)	O aplicativo como o usuário vê, com prints e vídeos: cadastro, mapa, registro, tempo real entre dois aparelhos, alerta, offline, relatórios, status e tema escuro
Distribuído	Script próprio + Docker	Balanceamento, queda de réplica, queda do broker, falha da Open-Meteo e rastreamento por X-Request-Id
Carga	autocannon	Vazão e latências p50/p95/p99 com 1 e com 2 réplicas
Qualidade	tsc, ESLint, expo lint	Tipos e padrões de código no backend, no app e nos testes

Fonte: elaborado pelos autores (2026).

4.8 Riscos e mitigação

A Tabela 7 relaciona os principais riscos identificados, a mitigação adotada e como ela foi verificada.

Tabela 7 – Riscos e mitigação

Risco	Mitigação adotada	Verificação
Queda do broker de mensagens	Outbox transacional, reconexão com backoff e reassinatura das filas	Teste distribuído 3
Queda de uma instância	2 réplicas, max_fails e proxy_next_upstream no Nginx	Teste distribuído 2

Risco	Mitigação adotada	Verificação
API externa lenta ou fora do ar	Timeout, circuit breaker e cache persistido	Teste distribuído 4
Celular sem conexão	Fila offline persistente e sincronização idempotente	E2E 08
Registros duplicados em reenvios	idempotencyKey única e inbox nos consumidores	Integração 4 e E2E 08
Eventos fora de ordem entre réplicas	Versão no retrato da ocorrência; projeções só aceitam versão maior	Testes unitários
Upload malicioso ou grande	Limite de 5 MB (serviço) e 8 MB (gateway); tipo pelos bytes iniciais	Integração (413 e 415)
Força bruta no login	10 tentativas por minuto por IP + e-mail	Integração (429)
Sem Android SDK no computador	App validado na web em viewport de celular, bundles nativos compilados e roteiro de teste no celular (COMO_TESTAR.md)	Build 08 e E2E

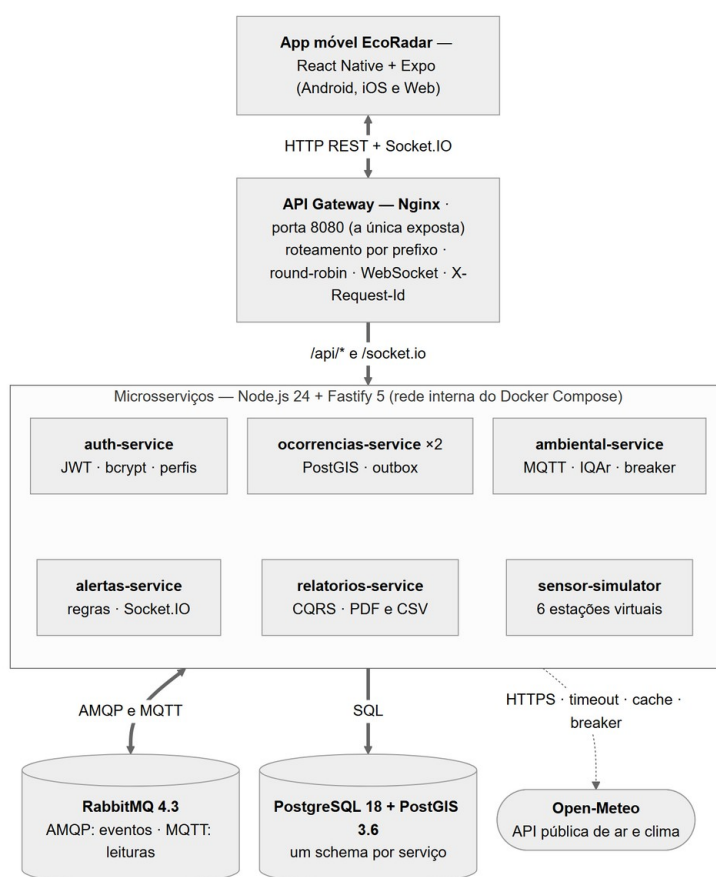
Fonte: elaborado pelos autores (2026).

5 PROJETO (ESTRUTURA E MÓDULOS) DO PROGRAMA

5.1 Arquitetura geral

A Figura 2 mostra a arquitetura em camadas. O aplicativo fala apenas com o gateway Nginx, única porta pública (8080), que encaminha cada prefixo de URL ao serviço responsável, distribui as chamadas de `/api/ocorrencias` entre as duas réplicas e mantém aberta a conexão WebSocket de `/socket.io`. Os seis microsserviços rodam numa rede interna do Docker e compartilham apenas a infraestrutura: o RabbitMQ, para eventos e leituras de sensores, e o PostgreSQL, em que cada serviço acessa somente o próprio schema. O código comum — cliente AMQP resiliente, envelopes de eventos, erros padronizados, autenticação, *backoff*, cache e o servidor Fastify base com `/health`, `/ready`, `X-Request-Id` e Swagger — fica no pacote `packages/shared`.

Figura 2 – Arquitetura do EcoRadar Urbano



Fonte: elaborado pelos autores (2026).

5.2 Microsserviços

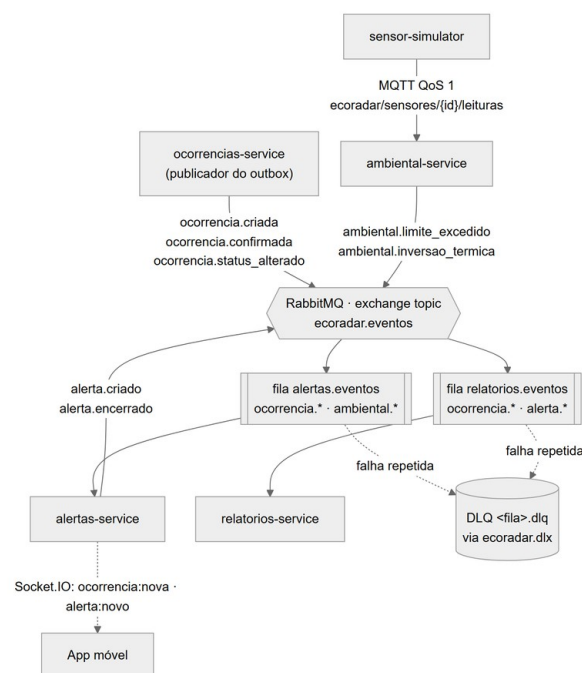
Os serviços somam 28 *endpoints* de negócio, todos documentados em OpenAPI (Swagger UI em `/api/<serviço>/docs`) a partir dos mesmos esquemas Zod que validam as requisições:

- **auth-service** — cadastro, login e perfis (POST `/api/auth/registrar`, POST `/api/auth/login`, GET/PATCH `/api/auth/me` e, para o administrador, GET `/api/auth/usuarios` e PATCH `/api/auth/usuarios/{id}/perfil`). Senhas com bcrypt (custo 10) e um *hash* “fantasma” para que o tempo de resposta não revele quais e-mails existem; JWT válido por 12 h. Não publica eventos. Schema `auth`;
- **ocorrencias-service** (2 réplicas) — registro, listagem, detalhe, confirmação e mudança de status (`/api/ocorrencias`, `/id`, `/id/confirmar`, `/id/status`), mananciais em GeoJSON, verificação de ponto e estado do outbox; serve as fotos em `/uploads`. Publica `ocorrencia.criada`, `ocorrencia.confirmada` e `ocorrencia.status_alterado` pelo outbox. Schema `ocorrencias`;
- **ambiental-service** — assina as leituras MQTT, calcula IQAr, inversão térmica e situação dos córregos, consulta a Open-Meteo e expõe estações, séries de leituras, resumo e estado das integrações. Publica `ambiental.limite_excedido` e `ambiental.inversao_termica`. Schema `ambiental`;
- **alertas-service** — consome `ocorrencia.*` e `ambiental.*`, aplica o motor de regras, grava os alertas, transmite-os por Socket.IO e publica `alerta.criado` e `alerta.encerrado`; lista, detalha e encerra alertas e expira os ativos há mais de 12 h. Schema `alertas`;
- **relatorios-service** — consome `ocorrencia.*` e `alerta.*` para manter a visão de leitura (CQRS) e oferece estatísticas e exportação em CSV e PDF. Schema `relatorios`;
- **sensor-simulator** — seis estações virtuais em bairros de São Paulo (Sé, Pinheiros, Ipiranga, Itaquera, Santana e Santo Amaro) que publicam por MQTT a cada 5 s; o administrador pode forçar cenários de alagamento, poluição crítica e inversão térmica. Não usa banco.

5.3 Comunicação por eventos

A Figura 3 detalha quem publica e quem consome cada evento. Todos os eventos usam o mesmo envelope — identificador, tipo, versão, data, serviço e instância de origem, correlationId e dados — validado com Zod na chegada. As filas são duráveis e cada uma tem uma fila de mensagens mortas (<fila>.dlq) ligada à *exchange* ecoradar.dlx. O serviço de alertas processa uma mensagem por vez (*prefetch* 1), preservando a ordem das regras de deduplicação; o de relatórios recebe até 20 mensagens por vez (*prefetch* 20).

Figura 3 – Eventos publicados e consumidos

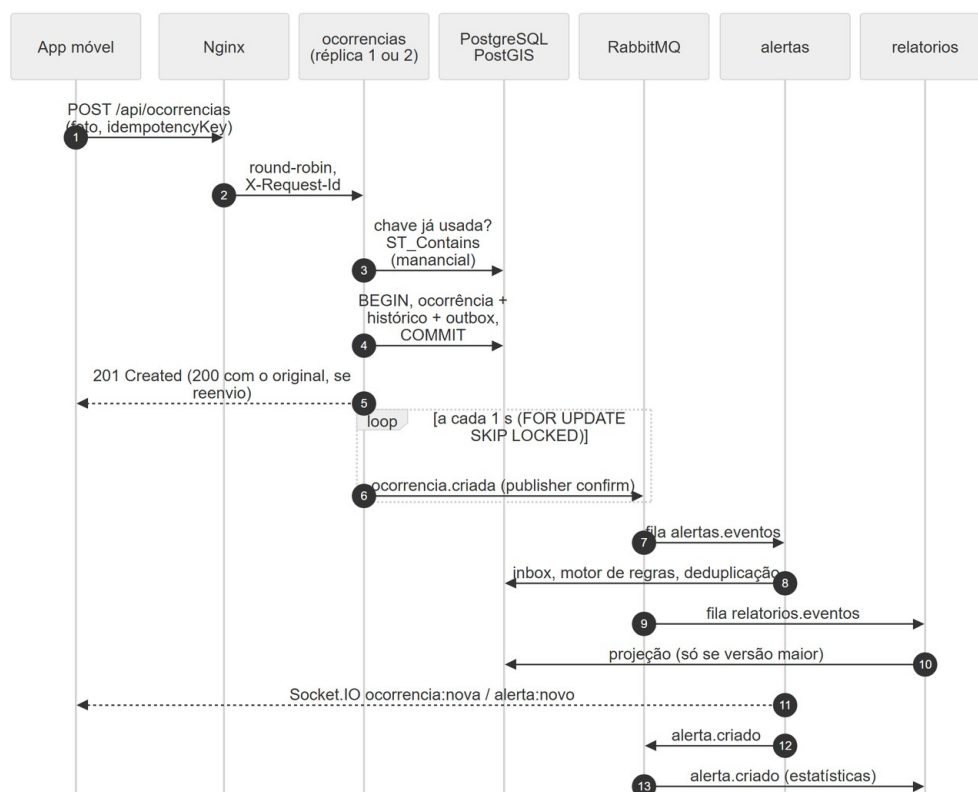


Fonte: elaborado pelos autores (2026).

5.4 Fluxos principais

A Figura 4 mostra o registro de uma ocorrência. A requisição passa pelo gateway, que escolhe uma réplica e acrescenta o X-Request-Id. A réplica verifica a chave de idempotência e o manancial, grava ocorrência, histórico e evento numa única transação e responde 201 (ou 200 com o registro original, se for um reenvio). O publicador do outbox envia o evento ao broker a cada segundo; alertas e relatórios o consomem de forma independente, e o serviço de alertas avisa todos os aparelhos conectados.

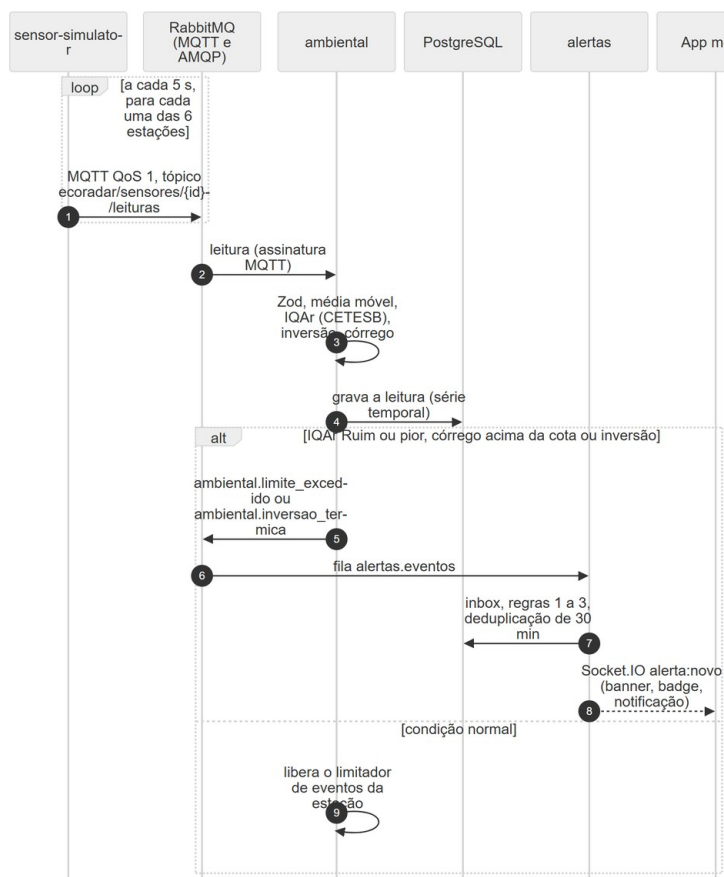
Figura 4 – Sequência do registro de uma ocorrência



Fonte: elaborado pelos autores (2026).

A Figura 5 mostra o caminho de uma leitura de sensor. O simulador publica por MQTT; o serviço ambiental valida a mensagem, atualiza a média móvel, calcula o IQAr, avalia inversão e córrego e grava a leitura. Se algum limite foi ultrapassado, publica um evento — no máximo um por estação e indicador por minuto, até a condição normalizar — e o serviço de alertas decide, pelas regras e pela deduplicação, se um novo alerta deve ser emitido.

Figura 5 – Sequência de uma leitura de sensor até o alerta

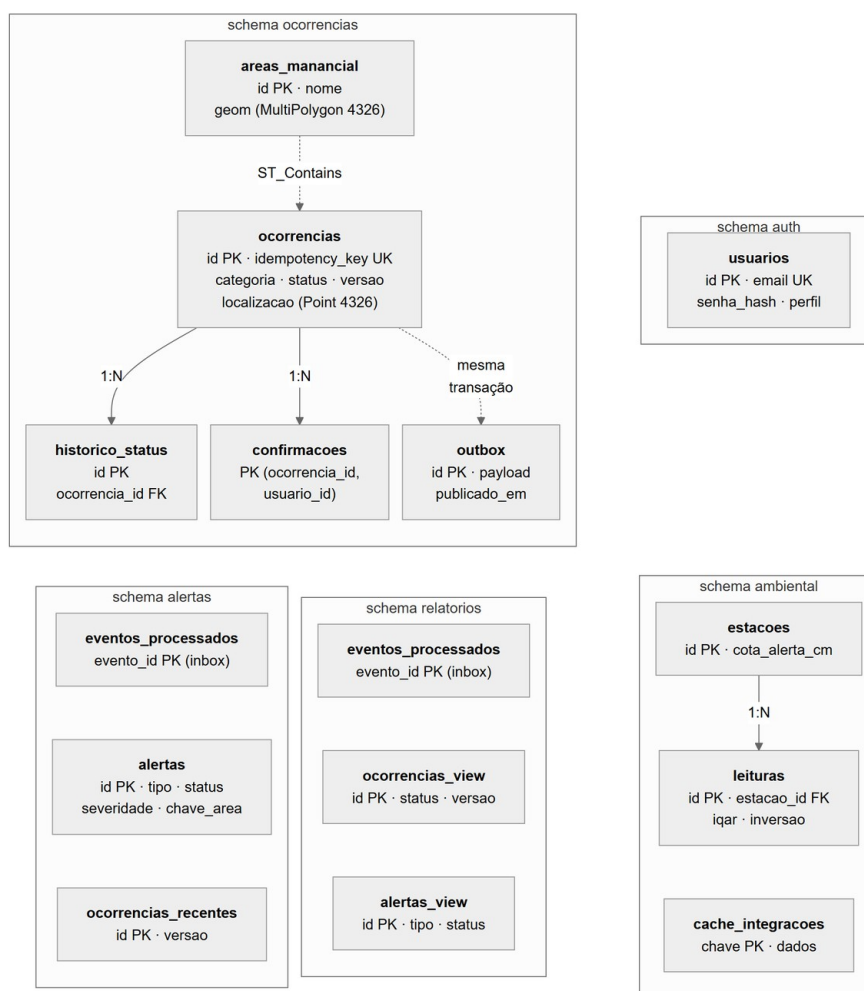


Fonte: elaborado pelos autores (2026).

5.5 Modelo de dados

O banco tem um schema e um usuário por serviço (Figura 6). Não há chaves estrangeiras entre schemas: a única ligação entre eles são os eventos. As chaves são UUID (exceto leituras, com identidade sequencial), as tabelas de *inbox* (eventos_processados) garantem o consumo idempotente e o campo versao impede que eventos atrasados sobrescrevam dados mais novos nas visões dos serviços de alertas e de relatórios.

Figura 6 – Modelo de dados simplificado por schema

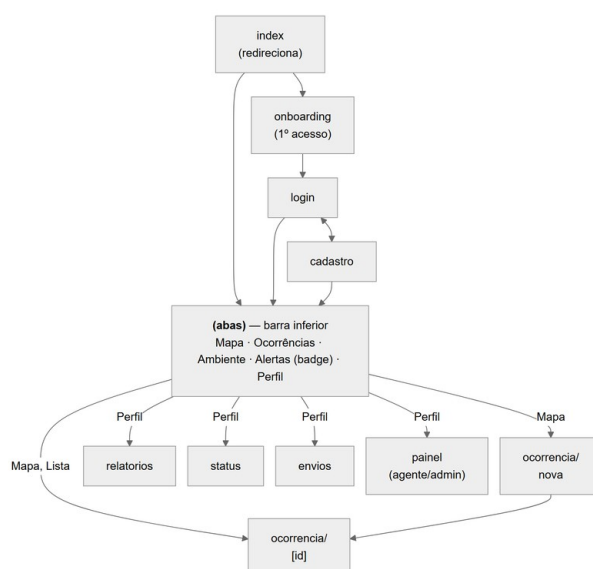


Fonte: elaborado pelos autores (2026).

5.6 Estrutura do aplicativo e navegação

O código do aplicativo fica em mobile/src: app contém as 14 telas do Expo Router; componentes, os componentes reutilizáveis, inclusive o mapa por plataforma; api, o cliente Axios e as consultas do TanStack Query; estado, os stores do Zustand (sessão, preferências, conexão e fila offline); serviços, tempo real, sincronização, localização, mídia, notificações e exportação, com variantes .web.ts quando necessário; além de tema e utils. A Figura 7 mostra a navegação: a rota inicial decide entre onboarding, login e abas; a partir das abas abrem-se o registro, o detalhe da ocorrência e, pelo perfil, relatórios, status do sistema, envios pendentes e o painel do agente.

Figura 7 – Navegação entre as telas do aplicativo



Fonte: elaborado pelos autores (2026).

5.7 Tratamento de erros e resiliência

- **respostas padronizadas:** todo erro segue o formato { erro: { codigo, mensagem, detalhes, requestId } }, com mensagens em português e detalhes por campo nas validações (422); o gateway também devolve JSON em 413 e 503;
- **validação e autorização:** esquemas Zod em todas as rotas; a verificação de perfil roda no *hook* onRequest, antes da validação do corpo; fotos são aceitas só até 5 MB e com tipo confirmado pelos bytes iniciais do arquivo;
- **mensageria:** reconexão AMQP e MQTT com *backoff* exponencial e *jitter* (1 s a 30 s), reassinatura automática das filas, *publisher confirms*, fila de mensagens mortas e consumidores idempotentes;
- **integração externa:** *timeout* de 5 s, *circuit breaker* e último valor válido persistido no banco;
- **aplicativo:** três tentativas com espera crescente em falhas transitórias, fila offline, cache persistido, *error boundary* para falhas de renderização e mensagens amigáveis por código HTTP;
- **operação:** *health checks* no Compose, encerramento gracioso ao receber SIGTERM e limpeza periódica do outbox e das tabelas de *inbox*.

6 RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA

6.1 Métricas do código

O projeto tem **16.920 linhas em 177 arquivos** de código-fonte e testes (15.504 não vazias), sem contar dependências, *builds* e arquivos gerados (Tabela 8). O aplicativo é o maior módulo (5.497 linhas); no backend, os serviços de ocorrências e ambiental concentram a lógica mais extensa.

Tabela 8 – Linhas de código por módulo

Módulo	Arquivos	Linhas	Não vazias
Pacote compartilhado	18	1.496	1.354
auth-service	6	486	444
ocorrencias-service	10	1.464	1.337
ambiental-service	15	1.603	1.453
alertas-service	6	859	782
relatorios-service	7	937	861
sensor-simulator	4	417	378
Seed (dados de demonstração)	5	647	598
Aplicativo móvel (mobile/src)	61	5.497	5.089
Testes (integração, E2E, distribuído, carga)	26	2.703	2.493
Scripts de automação	15	489	437
Gateway (Nginx) e infraestrutura	4	322	278
Total	177	16.920	15.504

Fonte: elaborado pelos autores (2026)

Por linguagem, predominam TypeScript (11.050 linhas) e TSX (3.868). Os trechos a seguir concentram as decisões de sistemas distribuídos; linhas omitidas aparecem como “(...)”.

6.2 Trechos selecionados

6.2.1 Motor de regras de alertas

O motor recebe um evento e devolve os alertas candidatos segundo cinco regras: IQAr ruim ou pior, córrego acima da cota, inversão térmica, três ou mais

ocorrências da mesma categoria em 1 km e 60 min e invasão ou desmatamento em manancial, com os limites centralizados no objeto PARAMETROS do mesmo arquivo. A Listagem 1 mostra a sexta regra, a deduplicação: um alerta do mesmo tipo, na mesma área e há menos de 30 min é suprimido — exceto quando a severidade aumentou, o que caracteriza escalonamento.

Listagem 1 – Deduplicação e escalonamento de alertas (backend/services/alertas-service/src/dominio/motor-regras.ts, linhas 212–227)

```
export function ehDuplicado(candidato: AlertaCandidato, existentes:
AlertaExistente[], agora: Date): boolean {
  return existentes.some((a) => {
    if (a.tipo !== candidato.tipo) return false;
    if (PESO_SEVERIDADE[candidato.severidade] >
PESO_SEVERIDADE[a.severidade]) return false;
    if (minutos(agora.getTime() - a.criadoEm.getTime()) >=
PARAMETROS.deduplicacaoJanelaMin) return false;
    if (candidato.chaveArea.startsWith('estacao:')) return a.chaveArea ===
candidato.chaveArea;
    if ((a.categoria ?? null) !== (candidato.categoria ?? null)) return false;
    if (a.latitude === null || a.longitude === null || candidato.latitude === null ||
candidato.longitude === null) {
      return a.chaveArea === candidato.chaveArea;
    }
    return (
      distanciaKm({ latitude: a.latitude, longitude: a.longitude }, { latitude:
candidato.latitude, longitude: candidato.longitude }) <=
      PARAMETROS.deduplicacaoRaioKm
    );
  });
}
```

6.2.2 Publicador do outbox

A Listagem 2 mostra o ciclo do publicador. A fonte trava as linhas pendentes com FOR UPDATE SKIP LOCKED, de modo que as duas réplicas nunca publicam o

mesmo evento; o lote é interrompido no primeiro erro para preservar a ordem, e as falhas reagendam o ciclo com *backoff* de 1 a 15 s.

Listagem 2 – Ciclo do publicador do outbox

(backend/services/ocorrencias-service/src/dominio/publicador-outbox.ts, linhas 54–69)

```
async executarCiclo(): Promise<{ publicados: number; falhas: number }> {
  if (!this.broker.conectado) return { publicados: 0, falhas: 0 };
  const resultado = await this.fonte.processarPendentes(this.opcoes.tamanhoLote
  ?? 50, async (linhas) => {
    const publicados: string[] = [];
    const falhas: ResultadoLote['falhas'] = [];
    for (const linha of linhas) {
      try {
        await this.broker.publicar(linha.payload);
        publicados.push(linha.id);
      } catch (erro) {
        falhas.push({ id: linha.id, erro: erro instanceof Error ? erro.message :
String(erro) });
        break;
      }
    }
    return { publicados, falhas };
  });
}
```

6.2.3 Idempotência na criação de ocorrências

Na Listagem 3, a chave enviada pelo aplicativo decide entre criar (201), devolver o original (200, com X-Idempotent-Replay: true) ou recusar uma chave de outro usuário (409). O segundo bloco trata a corrida entre réplicas: se as duas tentarem gravar a mesma chave, a restrição única do banco rejeita a segunda, que devolve o registro vencedor.

Listagem 3 – Decisão de idempotência e corrida entre réplicas

(backend/services/ocorrencias-service/src/rotas.ts, linhas 87–96 e 158–166)

```

const decisao = decidirIdempotencia(await repo.buscarPorIdempotencyKey(db,
dados.idempotencyKey), usuario.sub);
if (decisao.tipo === 'CONFLITO') {
  throw Erros.conflito('IDEMPOTENCY_KEY_EM_USO', 'Esta chave de
idempotência já foi usada por outro usuário.');
```

```

}
if (decisao.tipo === 'REPETIDA') {
  const original = await repo.buscarPorId(db, decisao.ocorrenciald);
  req.log.info({ ocorrenciald: decisao.ocorrenciald, idempotencyKey:
dados.idempotencyKey }, 'Reenvio idempotente: devolvendo registro original');
  reply.header('X-Idempotent-Replay', 'true');
  return reply.status(200).send(paraSnapshot(original!));
}
// (...)
if (ehViolacaoDeUnicidade(erro)) {
  const existente = await repo.buscarPorIdempotencyKey(db,
dados.idempotencyKey);
  const d = decidirIdempotencia(existente, usuario.sub);
  if (d.tipo === 'REPETIDA') {
    reply.header('X-Idempotent-Replay', 'true');
    return reply.status(200).send(paraSnapshot((await repo.buscarPorId(db,
d.ocorrenciald)!)));
  }
  throw Erros.conflito('IDEMPOTENCY_KEY_EM_USO', 'Esta chave de
idempotência já foi usada por outro usuário.');
```

```

}
```

6.2.4 Consultas PostGIS por manancial e por raio

A Listagem 4 traz as duas consultas espaciais. ST_Contains verifica se o ponto está em algum polígono de manancial; na listagem, ST_DWithin sobre geography filtra pelo raio em metros usando o índice GiST, e ST_Distance calcula a distância em quilômetros exibida no aplicativo.

Listagem 4 – Ponto-em-polígono e busca por raio (backend/services/ocorrencias-service/src/repositorio.ts, linhas 62–69 e 124–133)

```
export async function manancialNoPonto(db: Executor, lat: number, lon: number)
{
  const r = await db
    .select({ id: areasManancial.id, nome: areasManancial.nome })
    .from(areasManancial)
    .where(sql`ST_Contains(${areasManancial.geom}, ${ponto(lat, lon)})`)
    .limit(1);
  return r[0] ?? null;
}
// (...)
const temPonto = f.lat !== undefined && f.lon !== undefined;
const origem = temPonto ? ponto(f.lat!, f.lon!) : null;
if (origem && f.raioKm !== undefined) {
  // ST_DWithin sobre geography: distância em metros, usa o índice GiST
  condicoes.push(sql`ST_DWithin(${ocorrencias.localizacao}::geography, $
{origem}::geography, ${f.raioKm * 1000})`);
}
const where = condicoes.length ? and(...condicoes) : undefined;
const distancia = origem
  ? sql<number>`round((ST_Distance(${ocorrencias.localizacao}::geography, $
{origem}::geography) / 1000.0)::numeric, 3)`
  : sql<null>`NULL`;
```

6.2.5 Cálculo do IQAr

A Listagem 5 implementa a interpolação linear da Tabela 1 para um poluente (o trecho omitido extrapola acima da última faixa, com teto de 500); a função calcularIqar aplica o cálculo a cada poluente e adota o maior.

Listagem 5 – Índice de um poluente por interpolação na faixa
(backend/services/ambiental-service/src/dominio/iqar.ts, linhas 80–84 e 92–99)

```
export function indicePoluente(poluente: Poluente, concentracao: number):
IndicePoluente {
  const limites = LIMITES_CONCENTRACAO[poluente];
```

```

const c = Math.max(0, concentracao);
let faixa = limites.findIndex((lim) => c <= lim);
let indice: number;
// (...)
} else {
  const [ilni, iFin] = FAIXAS_INDICE[faixa];
  const clni = faixa === 0 ? 0 : limites[faixa - 1];
  const cFin = limites[faixa];
  indice = ilni + ((iFin - ilni) / (cFin - clni)) * (c - clni);
}
return { poluente, concentracao: c, indice: Math.round(indice), classe:
CLASSES[faixa] };
}

```

6.2.6 Circuit breaker da Open-Meteo

Na Listagem 6, o opossum abre o circuito com 50% de falhas (mínimo de três chamadas em 60 s) e testa a API de novo após 30 s; em falha, obter devolve o último valor com a fonte “cache” em vez de um erro.

Listagem 6 – Configuração do breaker e degradação graciosa
(backend/services/ambiental-service/src/integracoes/open-meteo.ts, linhas 99–106 e 221–224 e 230 e 237–238)

```

this.breaker = new CircuitBreaker(() => this.buscar(), {
  name: 'open-meteo',
  timeout: opcoes.timeoutMs,
  errorThresholdPercentage: 50,
  resetTimeout: opcoes.resetTimeoutMs ?? 30_000,
  volumeThreshold: opcoes.volumeThreshold ?? 3,
  rollingCountTimeout: 60_000,
});
// (...)
async obter(forcarAtualizacao = false): Promise<RespostaExterna> {
  if (!forcarAtualizacao && this.cache.obterValido(CHAVE)) return
this.resposta('ao_vivo', null);
  try {

```

```

    const dados = await this.breaker.fire();
  // (...)
  } catch (erro) {
  // (...)
    return this.cache.obterUltimo(CHAVE) ? this.resposta('cache', mensagem) :
    this.resposta('indisponivel', mensagem);
  }

```

6.2.7 Gateway Nginx: balanceamento e WebSocket

A Listagem 7 mostra o grupo de réplicas com *round-robin* (uma falha tira a réplica do rodízio por 5 s), a repetição da requisição na próxima réplica em erro de conexão ou 502/503 e o repasse dos cabeçalhos Upgrade e Connection que transformam a conexão HTTP em WebSocket.

Listagem 7 – Upstream com duas réplicas, repetição e proxy WebSocket
(gateway/nginx.conf, linhas 66–71 e 117–119 e 167–172)

```

upstream ocorrencias_service {
    zone ocorrencias_service 64k;
    server ocorrencias-service-1:3000 resolve max_fails=1 fail_timeout=5s;
    server ocorrencias-service-2:3000 resolve max_fails=1 fail_timeout=5s;
    keepalive 32;
}
# (...)
proxy_next_upstream error timeout http_502 http_503;
proxy_next_upstream_tries 2;
proxy_next_upstream_timeout 10s;
# (...)
location /socket.io/ {
    proxy_pass http://alertas_service;
    proxy_http_version 1.1;
    proxy_set_header Upgrade      $http_upgrade;
    proxy_set_header Connection   $connection_upgrade;
    proxy_set_header Host         $host;

```

6.2.8 Fila offline do aplicativo

Na Listagem 8, cada registro recebe uma chave UUID antes de sair do aparelho. Sem conexão, ou em falha de rede ou erro 502 ou superior, o item vai para a fila persistente e é reenviado depois com a mesma chave.

Listagem 8 – Registro com fila offline (mobile/src/servicos/sincronizacao.ts, linhas 42 –57)

```
export async function registrarOcorrencia(dados: DadosNovaOcorrencia, foto:
FotoLocal | null, online: boolean): Promise<ResultadoRegistro> {
  const item: ItemFila = { idempotencyKey: gerarId(), dados, foto, status:
'pendente', criadoEm: new Date().toISOString(), tentativas: 0 };
  const enfileirar = async (erro?: string): Promise<ResultadoRegistro> => {
    const guardado: ItemFila = { ...item, foto: foto ? await
guardarFotoParaFila(foto) : null, erro };
    useFilaOffline.getState().adicionar(guardado);
    return { tipo: 'na_fila', item: guardado };
  };
  if (!online) return enfileirar();
  try {
    return { tipo: 'enviada', ocorrencia: await enviarOcorrencia(item) };
  } catch (e) {
    const erro = paraErroApi(e);
    if (erro.ehDeRede || (erro.status !== null && erro.status >= 502)) return
enfileirar(erro.message);
    throw erro;
  }
}
```

6.2.9 Componente de mapa por plataforma

A Listagem 9 é o contrato comum às duas implementações do mapa: as telas importam componentes/mapa/Mapa e o Metro resolve Mapa.native.tsx (react-native-maps) no celular e Mapa.web.tsx (Leaflet) no navegador.

Listagem 9 – Contrato único do mapa para as duas plataformas
(mobile/src/componentes/mapa/Mapa.d.ts, linhas 1–8)

```
// Tipagem comum das implementações por plataforma:  
// Mapa.native.tsx (react-native-maps) e Mapa.web.tsx (react-leaflet +  
OpenStreetMap).  
// O Metro escolhe o arquivo certo pela extensão; o TypeScript usa esta  
declaração.  
import type { ComponentType } from 'react';  
import type { PropsMapa } from './tipos';  
  
declare const Mapa: ComponentType<PropsMapa>;  
export default Mapa;
```

7 APRESENTAÇÃO DO PROGRAMA EM FUNCIONAMENTO

7.1 Como as evidências foram produzidas

Todas as imagens desta seção foram geradas por execuções reais registradas em registros/. O aplicativo foi exportado para a web (expo export), servido pelo gateway e operado pelo Playwright com emulação do Pixel 7 (roteiro completo) e do iPhone 14 (telas principais), com geolocalização simulada em São Paulo. Os prints de infraestrutura, os logs dos testes de falha e os relatórios exportados vieram dos scripts de teste. Os **18 vídeos** das execuções, o relatório HTML do Playwright (registros/testes/e2e/relatorio-html/), o relatório de cobertura e a galeria completa com 70 prints (registros/index.html) acompanham o trabalho.

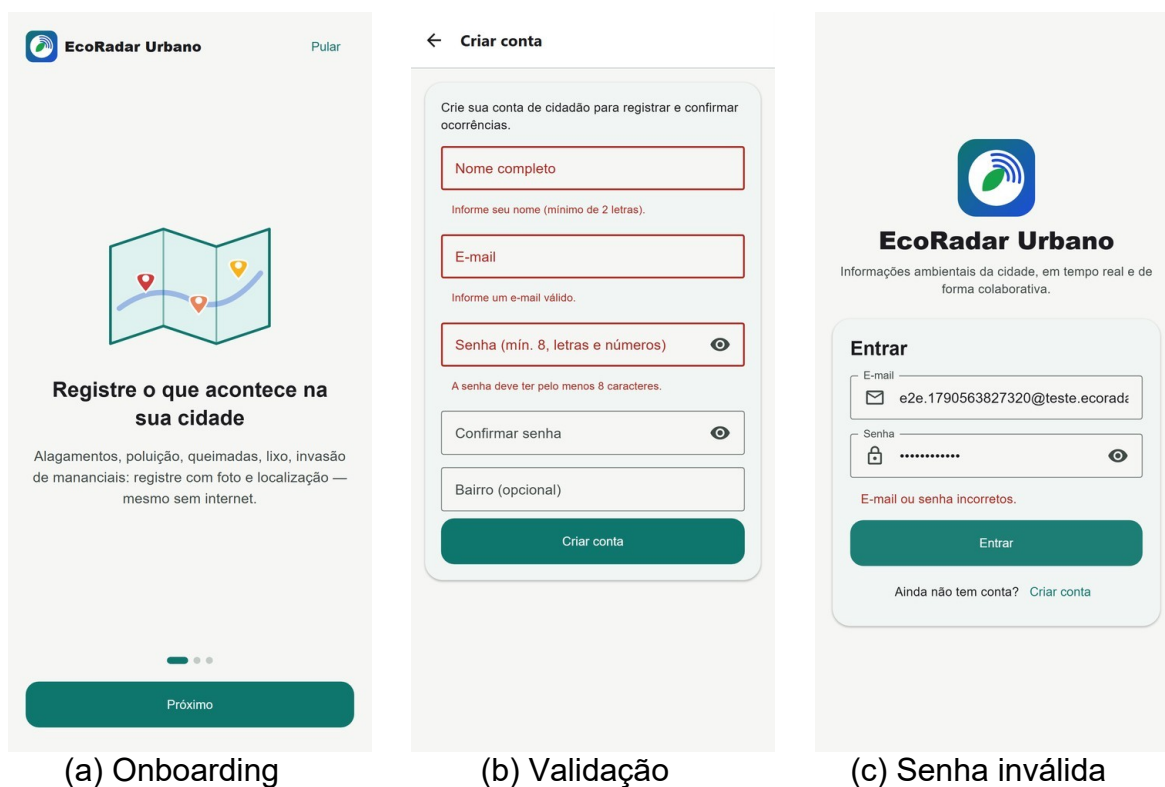
O computador de desenvolvimento não tinha Android SDK nem emulador; por isso os prints são do aplicativo web em *viewport* de celular. O código nativo foi verificado com a compilação dos pacotes Android e iOS (registros/build/08_expo-export-nativo.log), e o roteiro de teste no celular com o Expo Go está no COMO_TESTAR.md.

Além das funções básicas de registro e consulta, o grupo destaca como **funções extras**: tempo real entre aparelhos, alertas automáticos com deduplicação e escalonamento, registro offline sem duplicidade, aviso de área de manancial, confirmação colaborativa, painel do agente, integração com dados reais da Open-Meteo, relatórios em PDF e CSV, tela de status da arquitetura, simulador de cenários, tema escuro e documentação OpenAPI de todos os serviços.

7.2 Acesso, cadastro e tratamento de erros

Na Figura 8, o onboarding apresenta o aplicativo no primeiro acesso; o cadastro valida cada campo antes do envio (nome, e-mail e senha com no mínimo oito caracteres, letras e números); e o login com senha errada mostra a mensagem do servidor sem revelar se o e-mail existe.

Figura 8 – Onboarding, validação do cadastro e erro de login

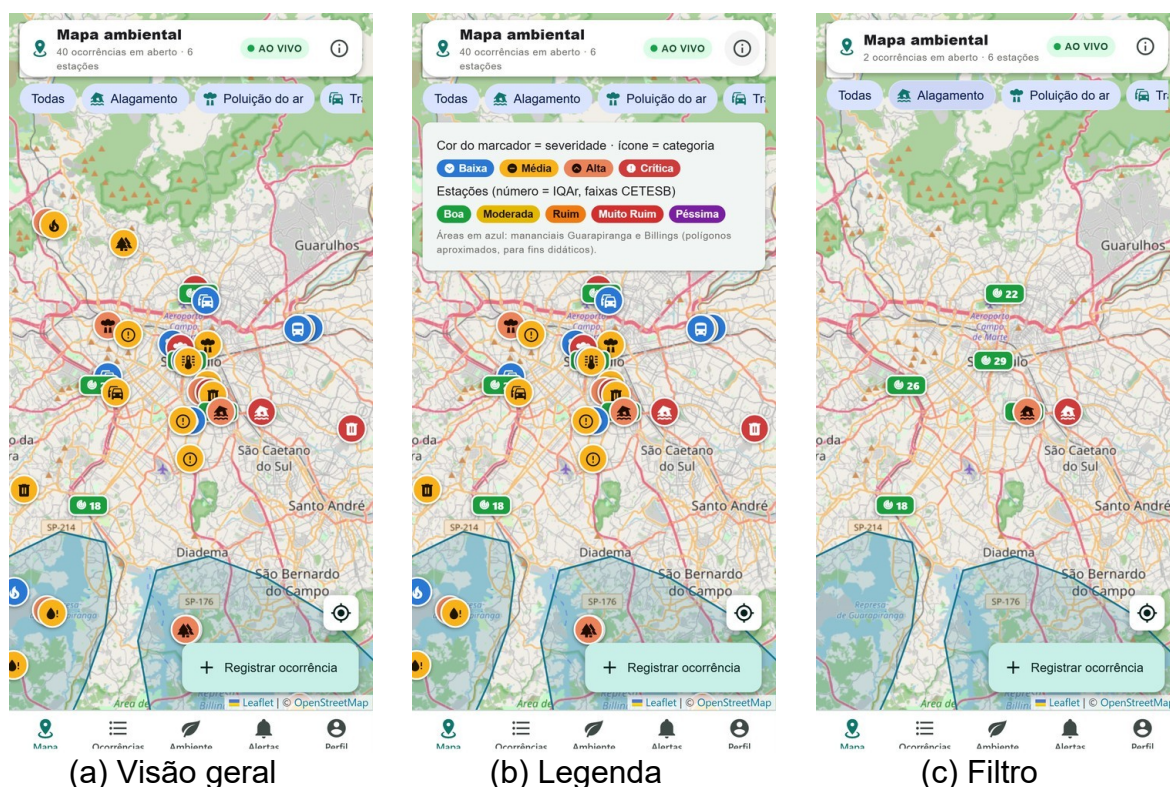


Fonte: elaborado pelos autores (2026).

7.3 Mapa ambiental

O mapa (Figura 9) mostra as ocorrências com cor pela severidade e ícone pela categoria, as estações com o valor do IQAr na cor da faixa CETESB e os polígonos dos mananciais Guarapiranga e Billings. A legenda explica a codificação e os filtros por categoria atualizam o mapa na hora; o selo "AO VIVO" indica a conexão em tempo real.

Figura 9 – Mapa com ocorrências, estações e mananciais; legenda; filtro por alagamento



Fonte: elaborado pelos autores (2026).

7.4 Registro de ocorrência com foto, GPS e manancial

A Figura 10 mostra o registro de uma invasão de manancial: categoria, severidade, descrição e foto; a localização obtida pelo GPS e ajustável no mapa, com o aviso de que o ponto está na área de proteção da Guarapiranga; e o detalhe logo após o envio, já com o banner do alerta prioritário gerado pela regra de manancial.

Figura 10 – Formulário, aviso de manancial e alerta prioritário após o registro

(a) Formulário

← Registrar ocorrência

1. O que está acontecendo?

Alagamento Poluição do ar Trânsito

Transporte público Invasão de manancial Desmatamento

Inversão térmica Queimada Descarte irregular de lixo

Outros

2. Qual a gravidade?

Baixa Média Alta Crítica

3. Descreva

Descrição

E2E: construção irregular na margem da represa 778c9a07

55/1000

4. Foto (opcional)

(b) Local e aviso

← Registrar ocorrência

5. Onde?

Usando sua localização atual. Toque no mapa ou arraste o pino para ajustar.

Mapa mostrando a localização da represa e a área de proteção.

-23.76500, -46.77000

Minha localização

Este ponto está dentro da área de proteção do manancial **Represa Guarapiranga**. Ocorrências de invasão ou desmatamento aqui geram alerta prioritário.

➤ Registrar ocorrência

(c) Registrada

NOVO ALERTA - ALTA

Invasão de manancial em área de manancial

Ocorrência de invasão de manancial registrada dentro da área de proteção Represa ...

Invasão de manancial

Região Represa Guarapiranga · agora mesmo

Severidade alta Aberta

Dentro da área de proteção do manancial Represa Guarapiranga.

E2E: construção irregular na margem da represa 778c9a07

Registrada por Elisa Nakamura

27/09/2026, 23:50

-23.76500, -46.77000

0 confirmações de outros cidadãos

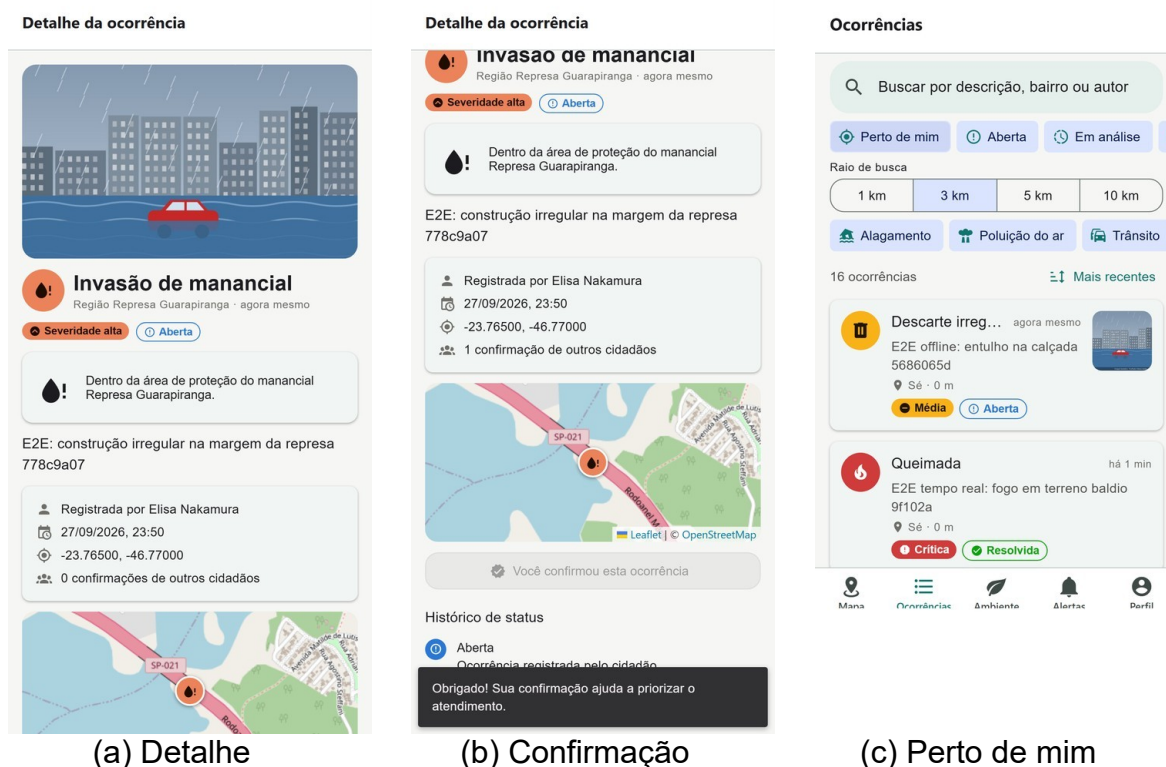
Ocorrência registrada com sucesso! Obrigado por colaborar.

Fonte: elaborado pelos autores (2026).

7.5 Detalhe, confirmação colaborativa e busca por proximidade

No detalhe (Figura 11) aparecem a foto, o autor, as coordenadas, o mapa e o histórico de status. Outro cidadão pode confirmar a ocorrência uma única vez, o que aumenta o contador usado para priorizar o atendimento. A lista “perto de mim”, capturada no iPhone 14, usa a consulta por raio do PostGIS e mostra a distância de cada ocorrência.

Figura 11 – Detalhe, confirmação colaborativa e lista por raio (iPhone 14)

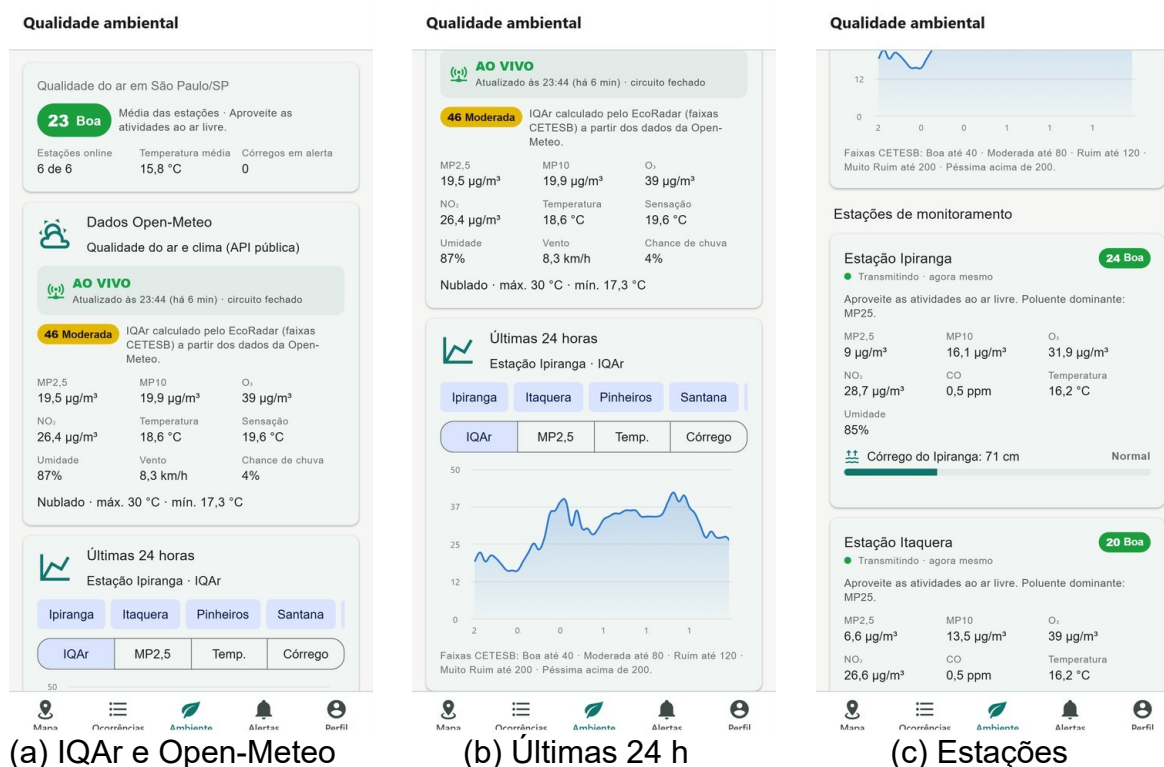


Fonte: elaborado pelos autores (2026).

7.6 Qualidade ambiental

A aba Ambiente (Figura 12) reúne o IQAr médio das estações, os dados da Open-Meteo (identificados como “ao vivo” ou “cache”, com o estado do circuito) com o IQAr recalculado pelas faixas da CETESB, o gráfico das últimas 24 horas por estação e indicador e os cartões de cada estação, com poluentes, temperatura e nível do córrego.

Figura 12 – IQAr e Open-Meteo, gráfico de 24 h e estações

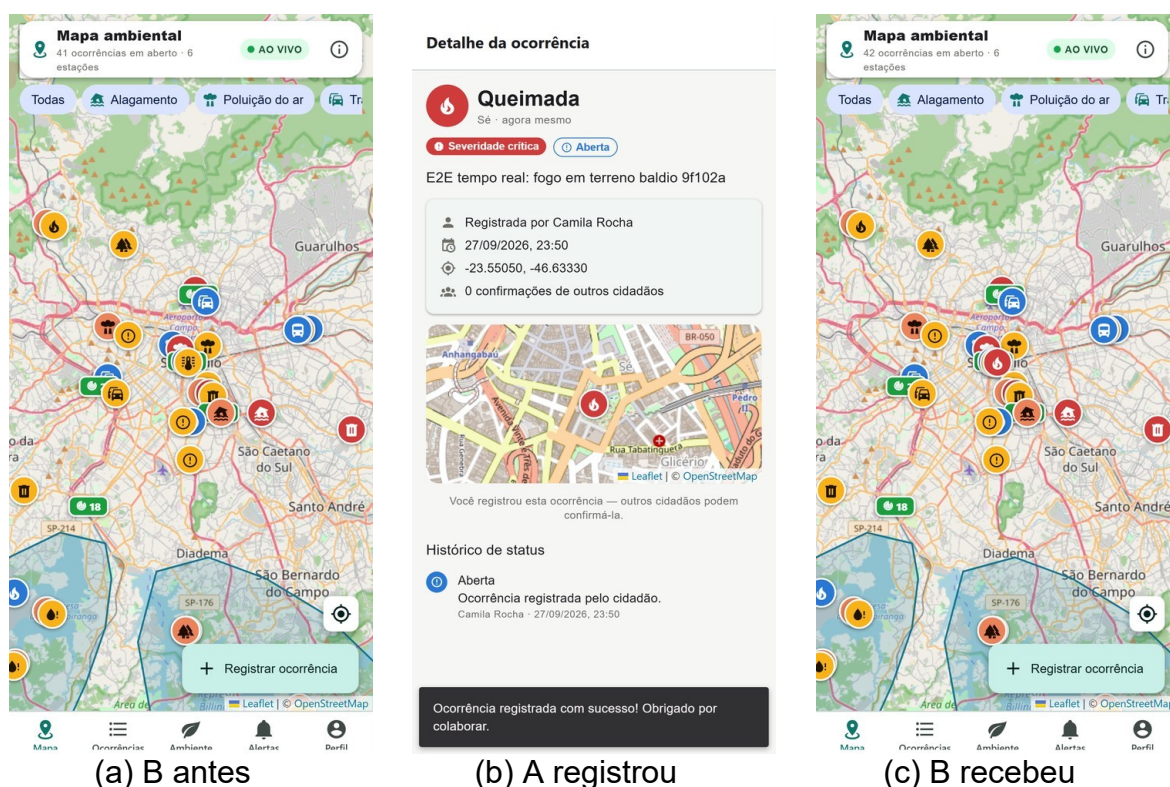


Fonte: elaborado pelos autores (2026).

7.7 Tempo real entre dois dispositivos

Na Figura 13, dois contextos de navegador independentes representam dois aparelhos. O dispositivo B mostra 41 ocorrências em aberto; o dispositivo A registra uma queimada; sem recarregar a tela, o dispositivo B passa a mostrar 42 ocorrências e o novo marcador, recebido pelo evento `ocorrencia:nova` do Socket.IO.

Figura 13 – Ocorrência registrada no dispositivo A aparece no dispositivo B

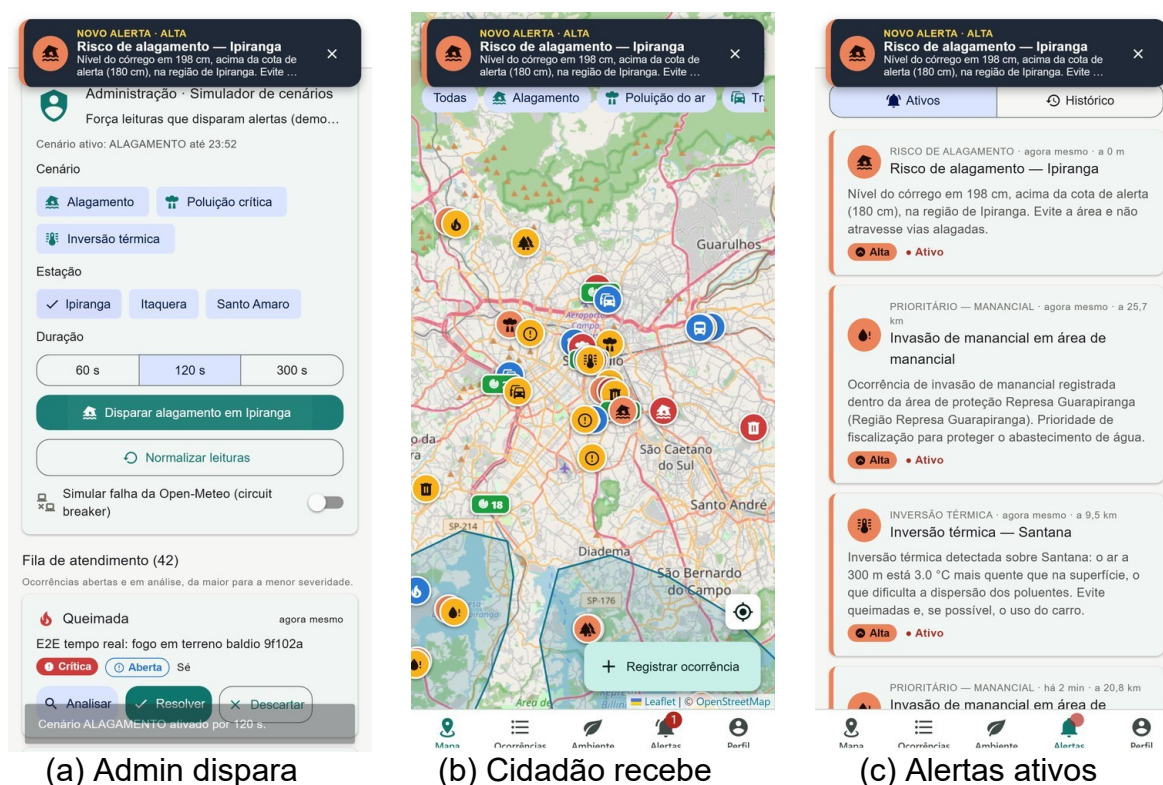


Fonte: elaborado pelos autores (2026).

7.8 Alerta disparado

Na Figura 14, o administrador dispara pelo painel o cenário de alagamento na estação Ipiranga por 120 s. O simulador eleva o nível do córrego acima da cota de 180 cm, o serviço ambiental publica o evento, o motor de regras cria o alerta “Risco de alagamento — Ipiranga” e o cidadão, em outro aparelho, recebe o banner e o *badge* na aba Alertas, onde o alerta aparece entre os ativos com severidade e distância.

Figura 14 – Cenário disparado pelo administrador e alerta recebido pelo cidadão

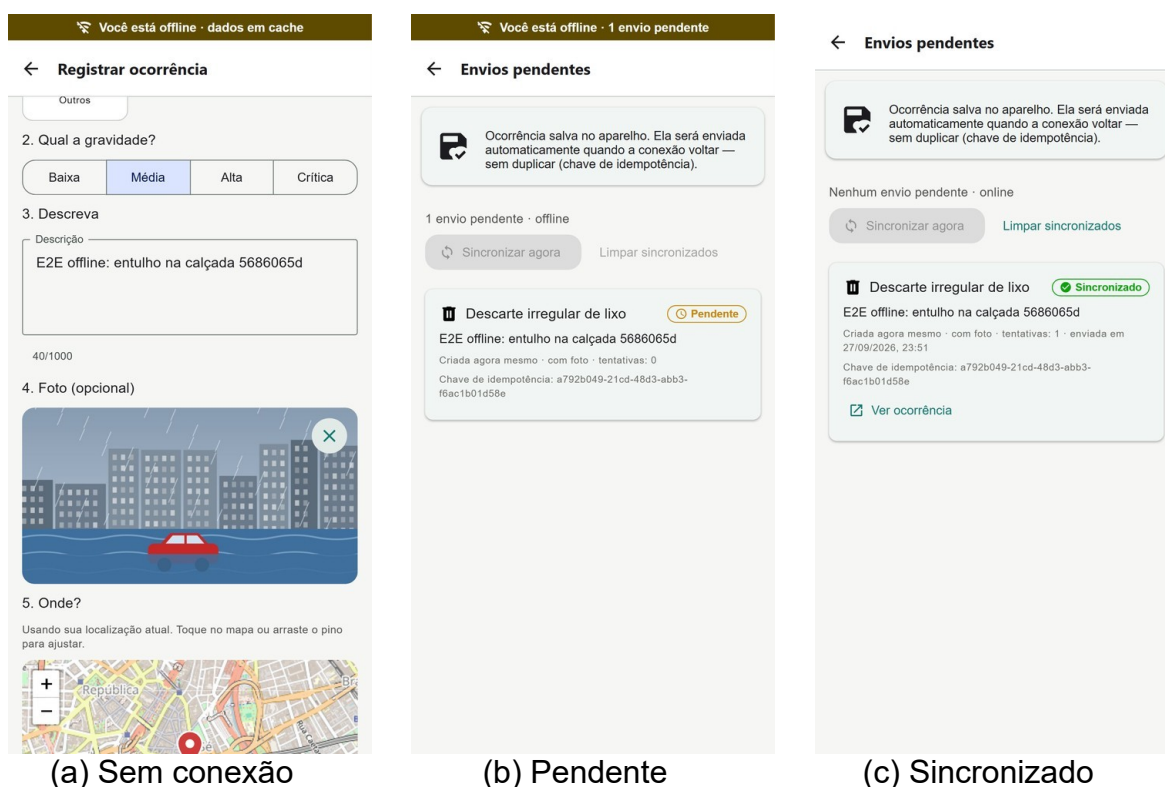


Fonte: elaborado pelos autores (2026).

7.9 Modo offline e sincronização

Na Figura 15, a conexão do navegador é desligada: a faixa “Você está offline” aparece, a ocorrência é salva no aparelho e fica pendente na tela de envios, com sua chave de idempotência. Ao religar a conexão, o envio é sincronizado automaticamente; o teste reenviou em seguida a mesma chave e confirmou que o servidor devolveu o registro original, sem duplicar.

Figura 15 – Registro sem conexão, envio pendente e sincronização automática

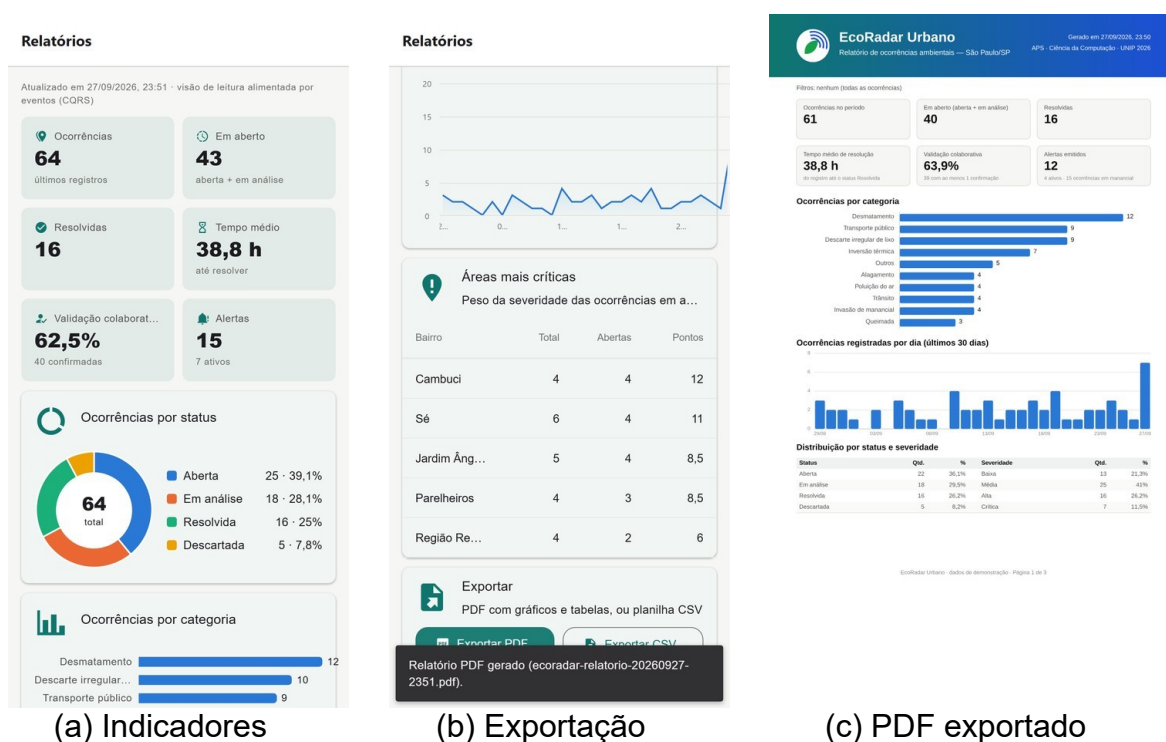


Fonte: elaborado pelos autores (2026).

7.10 Relatórios exportados

A tela de relatórios (Figura 16) mostra indicadores (ocorrências, em aberto, resolvidas, tempo médio de resolução, taxa de validação colaborativa e alertas), gráficos por status e categoria e as áreas mais críticas, com exportação em PDF e CSV. O PDF é gerado no servidor com os mesmos dados; a primeira página do arquivo exportado no teste de integração aparece ao lado.

Figura 16 – Indicadores, exportação e primeira página do PDF gerado

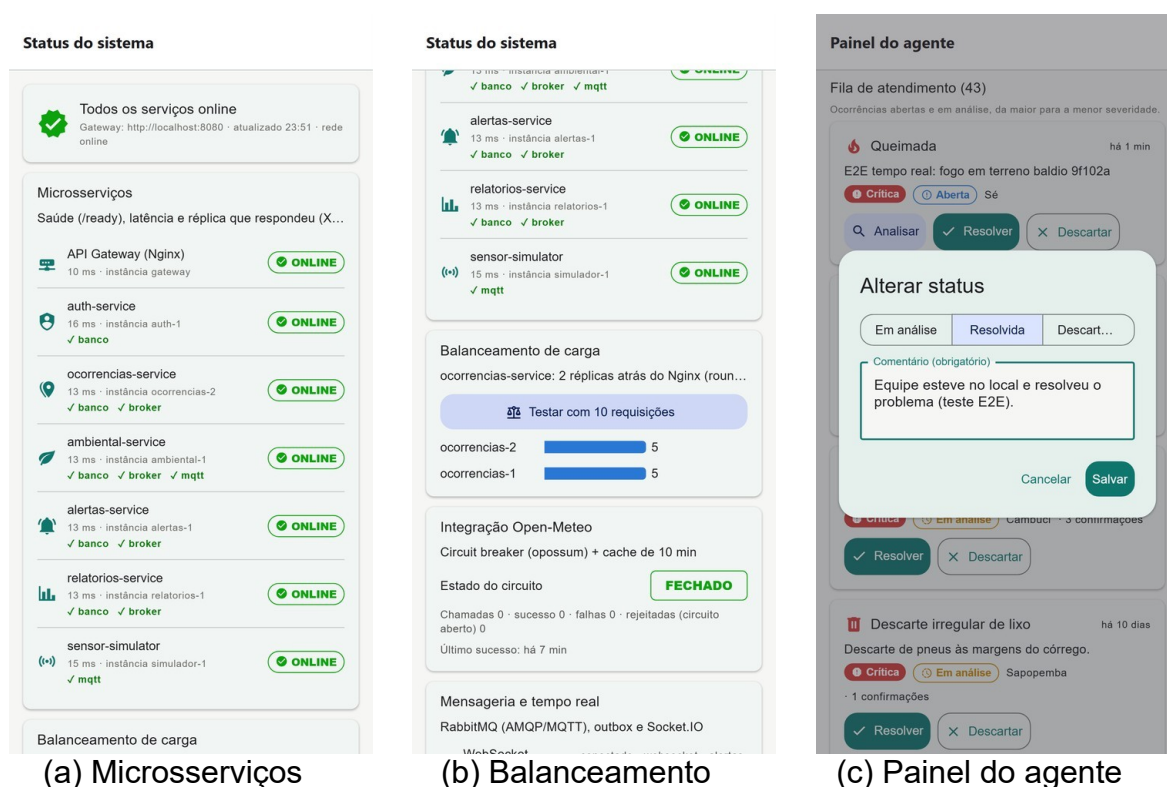


Fonte: elaborado pelos autores (2026).

7.11 Status do sistema e painel do agente

A tela de status (Figura 17) torna a arquitetura visível: saúde e latência de cada serviço e a instância que respondeu, dependências (banco, broker, MQTT), teste de balanceamento com dez requisições (cinco em cada réplica), estado do *circuit breaker*, conexões WebSocket e MQTT e o estado do outbox. O painel do agente ordena a fila de atendimento por severidade e permite analisar, resolver ou descartar, sempre com comentário.

Figura 17 – Status dos microsserviços, balanceamento e painel do agente

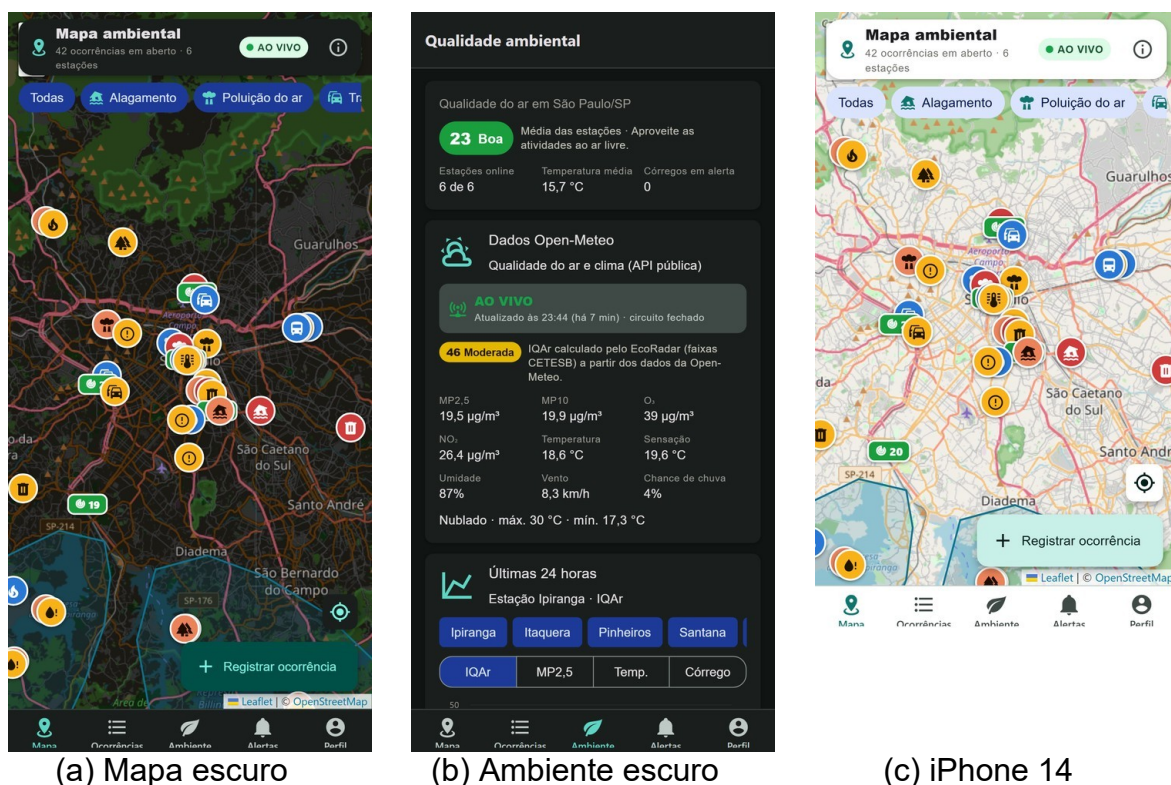


Fonte: elaborado pelos autores (2026).

7.12 Tema escuro e outro aparelho

O tema escuro (Figura 18) pode seguir o sistema ou ser escolhido no perfil, e alcança também o mapa e os gráficos. A mesma navegação foi repetida com a emulação do iPhone 14.

Figura 18 – Tema escuro no mapa e na qualidade ambiental; mapa no iPhone 14



Fonte: elaborado pelos autores (2026).

7.13 Testes de tolerância a falhas

O script de testes distribuídos manipula os contêineres com docker compose stop/start e verifica o comportamento pelo gateway. Os cinco cenários passaram (Tabela 9); os logs completos e as imagens estão em registros/testes/distribuido/.

Tabela 9 – Resultados dos testes de sistema distribuído

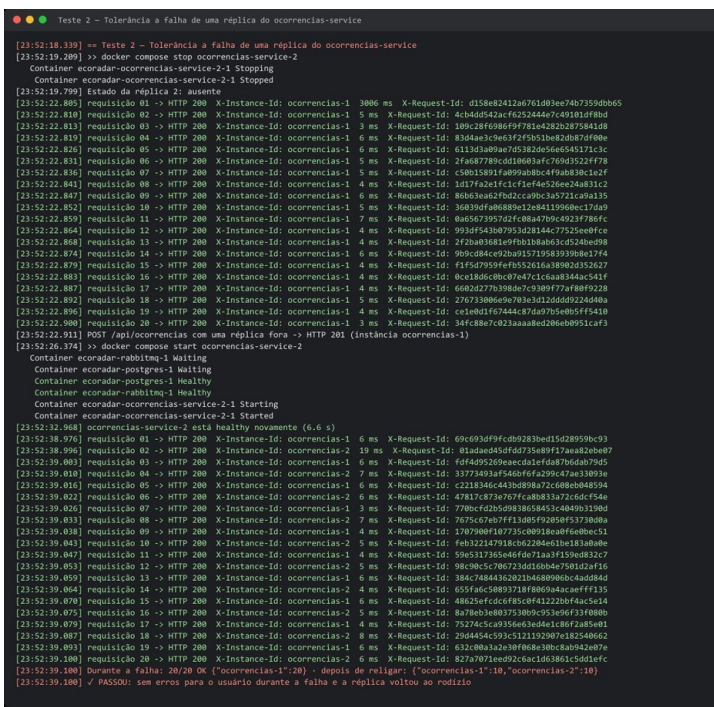
Teste	Resultado obtido
Balanceamento de carga	20 requisições: 10 em ocorrencias-1 e 10 em ocorrencias-2 (X-Instance-Id alternando)
Falha de réplica	Réplica 2 parada: 20 de 20 com sucesso, todas na réplica 1; religada, voltou ao rodízio (10/10)
Queda do broker	Broker fora por cerca de 20 s: 5 eventos ficaram pendentes no outbox; na volta, outbox zerado, relatórios de 65 para 70 ocorrências e alerta de manancial gerado
Falha da Open-Meteo	Circuito ABERTO e fonte “cache” durante a falha; FECHADO e “ao vivo” após o tempo de recuperação

Teste	Resultado obtido
Rastreamento	O mesmo X-Request-Id encontrado nos logs do gateway, da réplica 2, do serviço de alertas e do de relatórios

Fonte: elaborado pelos autores (2026), a partir de registros/testes/distribuido/resumo.md.

A Figura 19 mostra o log do teste de falha de réplica e a tela de status durante a falha, em que o teste de balanceamento encontra apenas a réplica 1. Na Figura 20, com o broker parado, a tela de status aponta AMQP e MQTT desconectados e cinco eventos pendentes no outbox; após o religamento, tudo volta a ficar conectado e o outbox é zerado. A Figura 21 mostra a falha simulada da Open-Meteo: o aplicativo continua exibindo os últimos dados, marcados como “dados em cache”, e o status indica o circuito aberto. O rastreamento aponta a ponta pelo X-Request-Id aparece na Figura 22.

Figura 19 – Queda de uma réplica: log do teste e status com uma réplica



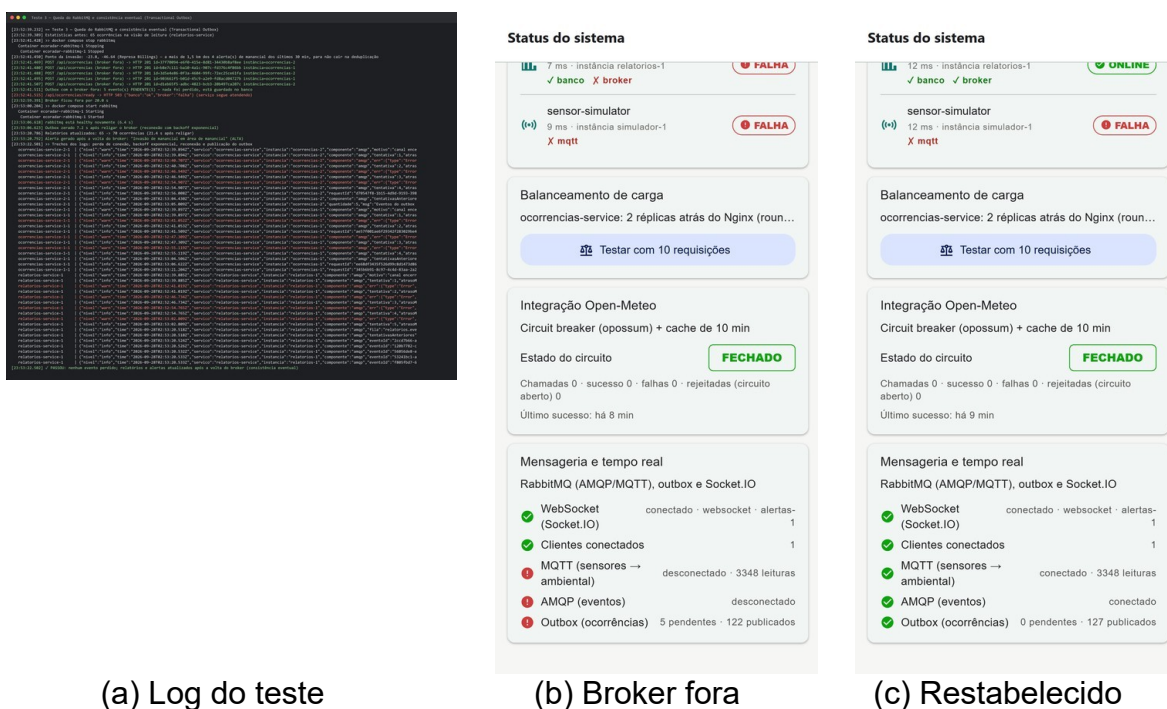


(a) Log do teste

(b) Status no app

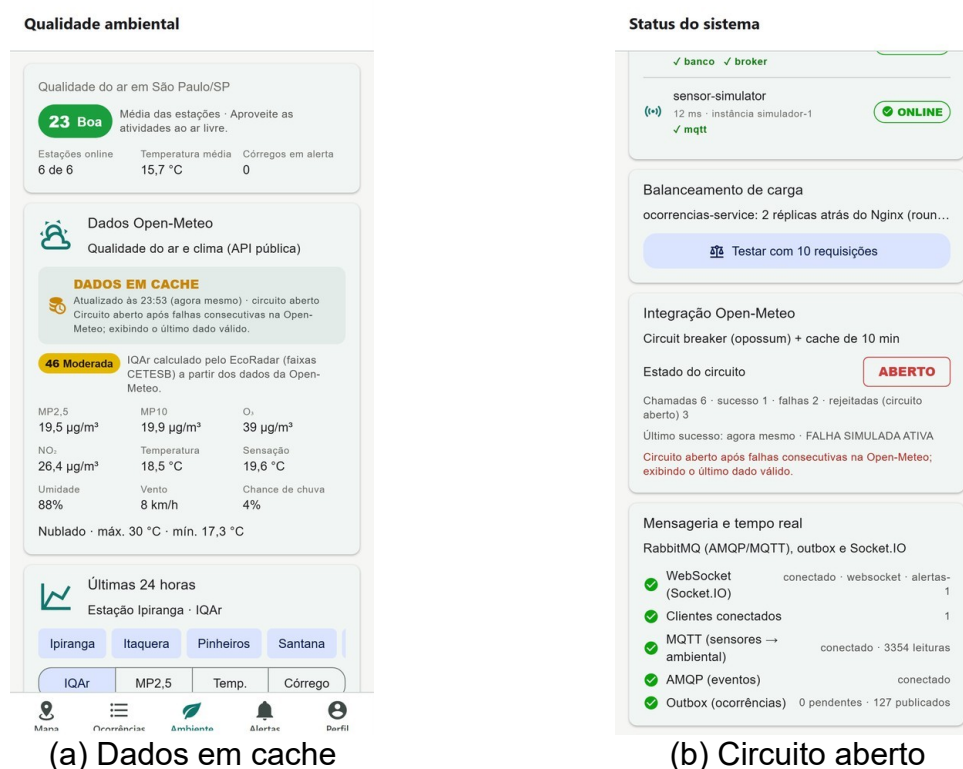
Fonte: elaborado pelos autores (2026).

Figura 20 – Queda do broker: status com o broker fora e após o restabelecimento



Fonte: elaborado pelos autores (2026).

Figura 21 – Circuit breaker: dados em cache e circuito aberto no aplicativo



Fonte: elaborado pelos autores (2026).

Figura 22 – Mesmo X-Request-Id nos logs de quatro contêineres

```

Teste 5 - Rastreamento distribuído pelo X-Request-Id
[23:53:57.108] == Teste 5 - Rastreamento distribuído pelo X-Request-Id
[23:53:57.184] POST /api/ocorrencias com X-Request-Id: rastreo-1798564037175-ff7ead1b -> HTTP 201; resposta devolveu X-Request-Id=rastreo-1798564037175-ff7ead1b
[23:54:03.883] Linhas de log com o ID: 6 - servicos: relatorios-service-1, ocorrencias-service-2-1, alertas-service-1, gateway-1
relatorios-service-1 | {"nivel":"info","time":"2026-09-28T02:53:57.242Z","servico":"relatorios-service","instancia":"relatorios-1","componente":"amqp","eventoId":"58492185-5656-ocorrencias-service-2-1 | {"nivel":"info","time":"2026-09-28T02:53:57.178Z","servico":"ocorrencias-service","instancia":"ocorrencias-2","requestId":"rastreo-1798564037175-ff7ea
ocorrencias-service-2-1 | {"nivel":"info","time":"2026-09-28T02:53:57.183Z","servico":"ocorrencias-service","instancia":"ocorrencias-2","requestId":"rastreo-1798564037175-ff7ea
ocorrencias-service-2-1 | {"nivel":"info","time":"2026-09-28T02:53:57.183Z","servico":"ocorrencias-service","instancia":"ocorrencias-2","requestId":"rastreo-1798564037175-ff7ea
alertas-service-1 | {"nivel":"info","time":"2026-09-28T02:53:57.237Z","servico":"alertas-service","instancia":"alertas-1","componente":"amqp","eventoId":"58492185-5656-4986-9256
gateway-1 | {"time":"2026-09-28T02:53:57+00:00","servico":"gateway","requestId":"rastreo-1798564037175-ff7ead1b","cliente":"172.18.0.1","metodo":"POST","url":"/api/ocorrencias"
[23:54:03.883] ✓ PASSOU: o mesmo ID aparece no gateway, no serviço que atendeu e nos consumidores do evento

```

Fonte: elaborado pelos autores (2026).

7.14 Resultados dos testes

A Tabela 10 consolida os resultados da execução completa registrada em 27/09/2026. Todos os testes passaram. A cobertura de linhas das regras de negócio ficou em 97,47%, bem acima da meta de 70% definida para o projeto; os 12 cenários E2E que aparecem como “ignorados” são o roteiro completo, configurado para rodar só no Pixel 7 — no iPhone 14 roda a navegação pelas telas principais.

Tabela 10 – Resumo dos testes executados

Tipo	Resultado
Unitários (Vitest)	105 testes em 10 arquivos, todos aprovados; cobertura de linhas 97,47% (501/514), instruções 95,15%, funções 89,92% e ramos 88,6%
Integração (via gateway)	20 de 20 aprovados, incluindo os erros 401, 403, 404, 409, 413, 415, 422 e 429
Ponta a ponta (Playwright)	16 de 16 aprovados (Pixel 7 e iPhone 14), 70 prints e 18 vídeos
Sistema distribuído	5 de 5 cenários aprovados
Qualidade de código	tsc e ESLint sem erros no backend, no app e nos testes; expo-doctor 21/21

Fonte: elaborado pelos autores (2026), a partir de registros/RELATORIO_DE_TESTES.md.

7.15 Teste de carga: uma e duas réplicas

O autocannon disparou GET /api/ocorrencias pelo gateway com 50 conexões simultâneas durante 30 s, após 5 s de aquecimento, primeiro com uma réplica e depois com duas (Tabela 11 e Figura 23). Com duas réplicas a vazão subiu **69,5%**, de 1.474 para 2.498 requisições por segundo, e a latência mediana caiu de 33 para 20 ms, sem erros. O ganho não chega a 100% porque banco, broker, réplicas e

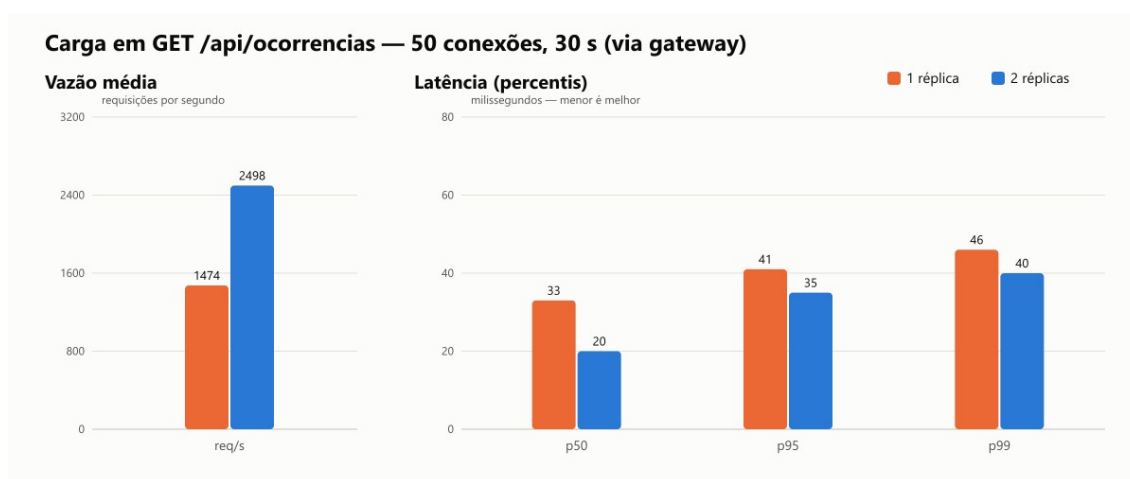
gerador de carga dividem o mesmo computador; os números servem para comparar as configurações, não como capacidade absoluta.

Tabela 11 – Teste de carga com 1 e 2 réplicas (latências em ms)

Cenário	Requisições	Req/s	p50	p95	p99	Máx.	Erros
1 réplica	44.209	1.474	33	41	46	226	0
2 réplicas	74.916	2.498	20	35	40	205	0

Fonte: elaborado pelos autores (2026), a partir de registros/testes/carga/resultado.md.

Figura 23 – Vazão e latências com 1 e 2 réplicas

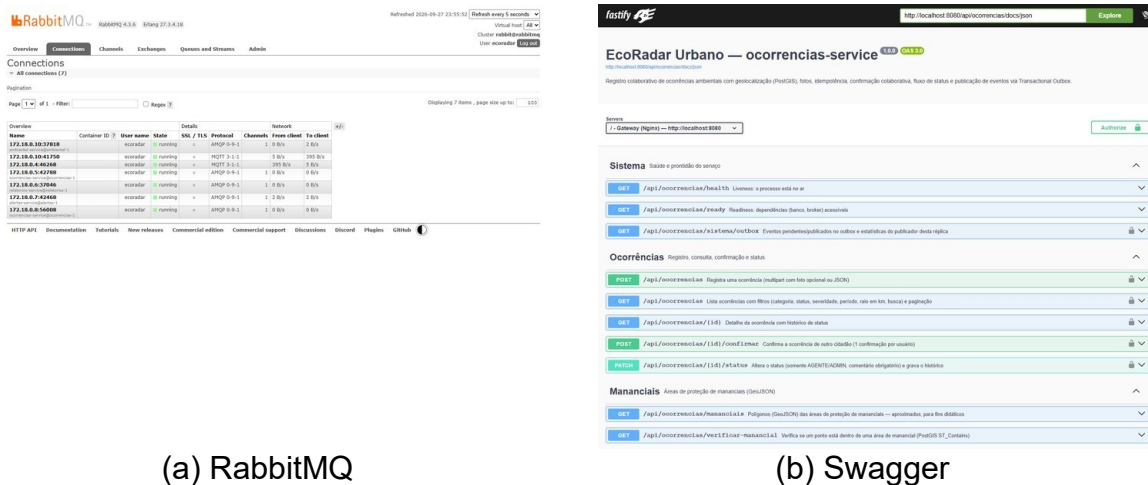


Fonte: elaborado pelos autores (2026).

7.16 Infraestrutura em execução

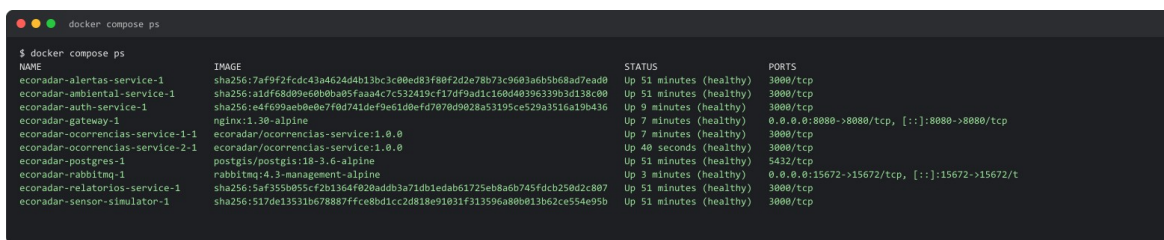
A Figura 24 mostra o painel do RabbitMQ com as conexões AMQP dos serviços e as conexões MQTT do simulador e do serviço ambiental, e a documentação OpenAPI do serviço de ocorrências; a Figura 25 mostra os dez contêineres saudáveis do Docker Compose, com apenas as portas 8080 e 15672 publicadas.

Figura 24 – Conexões AMQP e MQTT no RabbitMQ e Swagger do serviço de ocorrências



Fonte: elaborado pelos autores (2026).

Figura 25 – Contêineres do Docker Compose em execução



Fonte: elaborado pelos autores (2026).

7.17 Defeitos encontrados pelos testes e corrigidos

O refinamento do sistema veio em boa parte dos próprios testes. A Tabela 12 resume os dez problemas encontrados e como foram corrigidos; o último foi um defeito do teste, e não do sistema.

Tabela 12 – Defeitos encontrados pelos testes e correções

Problema e como foi detectado	Correção
Cidadão recebia 422 em vez de 403 ao alterar status (teste de fumaça)	Autorização movida para o hook onRequest, antes da validação do corpo
Alerta “Péssima” suprimido após um “Ruim” (cenário do simulador)	Escalonamento: severidade maior não é tratada como repetição
Imagens Docker com 494 MB	Código somente leitura para o usuário do contêiner: 368 MB

Problema e como foi detectado	Correção
Migração sem SRID e chaves estrangeiras com schema errado	Tipo geometry(Point, 4326) próprio e pós-processamento das migrações
App web abria em branco	Aguardar a hidratação do estado persistido antes de renderizar
Duas telas de login na pilha após sair (E2E)	Reinício da pilha de navegação ao entrar e sair
6 erros de socket no teste de carga	keepalive_requests 10000 no Nginx; nova execução sem erros
Seletor de raio estourava 375 px; zoom inicial muito aberto (prints)	Quatro opções de raio e zoom 11
429 para usuários diferentes da mesma rede	Limite aplicado por IP + e-mail no preHandler, com teste de integração
Teste de queda do broker falhou uma vez (defeito do teste)	Ponto de manancial sorteado longe de alertas recentes

Fonte: elaborado pelos autores (2026)

8 CONSIDERAÇÕES FINAIS: FORMAÇÃO E INTERDISCIPLINARIDADE

O EcoRadar Urbano atingiu os objetivos propostos: o aplicativo registra e consulta informações ambientais com foto, GPS e mapa, funciona sem conexão, recebe alertas em tempo real, e o backend distribuído se manteve funcional nos cenários de falha testados. Mais do que uma lista de funcionalidades, o trabalho permitiu ver na prática por que cada padrão existe, pois quase todos foram motivados por uma falha concreta observada durante o desenvolvimento ou os testes.

8.1 Interdisciplinaridade

O projeto exigiu integrar conteúdos de várias disciplinas do curso:

- **Sistemas Distribuídos:** microsserviços, mensageria, consistência eventual, idempotência, outbox, CQRS, circuit breaker, replicação e balanceamento, aplicados e testados sob falha;
- **Redes de Computadores:** HTTP e REST, WebSocket, MQTT e AMQP sobre TCP, proxy reverso, DNS interno do Docker, portas, NAT (que motivou a correção do limite de login por IP + e-mail) e firewall para o teste no celular;
- **Banco de Dados:** modelagem, transações ACID, restrições de unicidade, índices GiST, consultas espaciais, migrações e isolamento por schema e usuário;
- **Engenharia de Software:** requisitos, casos de uso, desenvolvimento incremental, controle de versão, testes em vários níveis, integração de qualidade e documentação;
- **Programação para Dispositivos Móveis:** React Native, navegação, permissões, câmera, GPS, armazenamento seguro, notificações e design *offline-first*;
- **Estruturas de Dados e Algoritmos:** filas (outbox e fila offline), janelas de média móvel, cache com TTL, interpolação linear do IQAr e cálculo de distância geodésica;
- **Segurança da Informação:** *hash* de senhas, JWT, limitação de tentativas, validação de entrada, verificação do tipo real dos arquivos e menor privilégio no banco;

- **Computação Gráfica e Interface com o Usuário:** mapas, codificação por cor e ícone, gráficos, tema escuro, acessibilidade e paleta adequada a daltônicos.

Fora da computação, o tema trouxe conceitos de **Ciências Ambientais** (poluentes, padrões de qualidade do ar, proteção de mananciais), **Geografia e Geoprocessamento** (sistemas de referência, polígonos, análise por proximidade), **Meteorologia** (inversão térmica, dados de clima), **Urbanismo** (drenagem, ocupação de áreas de proteção, mobilidade), **Saúde Pública** (efeitos da poluição, recomendações associadas às faixas do IQAr) e **Políticas Públicas** (legislação federal e estadual de qualidade do ar e de mananciais, participação cidadã e transparência de dados).

8.2 Contribuição para a formação

Nós começamos o trabalho conhecendo os conceitos de sistemas distribuídos principalmente pela teoria. Ao construir o EcoRadar, percebemos que cada garantia tem um custo e um motivo: a idempotência só fez sentido quando vimos o mesmo registro chegar duas vezes depois de uma queda de rede; o outbox, quando paramos o broker no meio de um teste e precisamos provar que nenhum evento se perdeu; o limite de tentativas de login, quando descobrimos que ele bloqueava todos os colegas conectados na mesma rede.

Também aprendemos a tratar testes e evidências como parte do produto, e não como etapa final: vários defeitos só apareceram porque automatizamos cenários de falha, de carga e de uso em dois aparelhos ao mesmo tempo. Trabalhar com um domínio real — qualidade do ar, mananciais, alagamentos — mostrou que decisões técnicas afetam diretamente a utilidade da informação para a população. Levamos do projeto a prática de justificar escolhas tecnológicas, documentar decisões e medir resultados, habilidades que esperamos usar no estágio e na vida profissional.

8.3 Limitações e trabalhos futuros

- os prints foram gerados na versão web em *viewport* de celular, pois não havia Android SDK no computador de desenvolvimento; o teste em aparelho real está roteirizado no `COMO_TESTAR.md`;
- as notificações são locais, compatíveis com o Expo Go; notificações *push* remotas exigiriam um *development build* e um serviço de envio;

- as estações são simuladas e o IQAr usa média móvel curta em vez das médias oficiais de 1 h, 8 h e 24 h; uma evolução natural é consumir dados abertos de estações reais;
- os polígonos dos mananciais são aproximados; o uso real exigiria os limites oficiais das áreas de proteção;
- o teste de carga foi feito num único computador; uma avaliação de capacidade exigiria máquinas separadas e orquestração com escalonamento automático (por exemplo, Kubernetes);
- banco e broker são instâncias únicas; replicação do PostgreSQL e *cluster* do RabbitMQ eliminariam esses pontos únicos de falha, e *tracing* distribuído com OpenTelemetry ampliaria a observabilidade.

9 BIBLIOGRAFIA

ABNT — ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR ISO 37122**: cidades e comunidades sustentáveis: indicadores para cidades inteligentes. Rio de Janeiro: ABNT, 2020.

BRASIL. Conselho Nacional do Meio Ambiente. Resolução nº 491, de 19 de novembro de 2018. Dispõe sobre padrões de qualidade do ar. **Diário Oficial da União**: seção 1, Brasília, DF, 21 nov. 2018. Disponível em: <https://www legisweb.com.br/legislacao/?id=369516>. Acesso em: 28 set. 2026.

BRASIL. Lei nº 14.850, de 2 de maio de 2024. Institui a Política Nacional de Qualidade do Ar. **Diário Oficial da União**: seção 1, Brasília, DF, 3 maio 2024. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2023-2026/2024/lei/l14850.htm. Acesso em: 28 set. 2026.

BRASIL. Conselho Nacional do Meio Ambiente. Resolução nº 506, de 5 de julho de 2024. Estabelece padrões nacionais de qualidade do ar e fornece diretrizes para sua aplicação. **Diário Oficial da União**: seção 1, Brasília, DF, ed. 130, p. 133, 9 jul. 2024. Disponível em: <https://www.in.gov.br/web/dou/-/resolucao-n-506-de-5-de-julho-de-2024-570885907>. Acesso em: 28 set. 2026.

CETESB — COMPANHIA AMBIENTAL DO ESTADO DE SÃO PAULO. **Padrões de qualidade do ar**. São Paulo: CETESB, [2026]. Disponível em: https://www.cetesb.sp.gov.br/cetesb/qualidade_ambiental/ar/informacoes_basicas/padroes_de_qualidade_do_ar. Acesso em: 28 set. 2026.

COULOURIS, George et al. **Sistemas distribuídos**: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013.

DOCKER. **Docker Compose**. [S. l.], [2026]. Disponível em: <https://docs.docker.com/compose/>. Acesso em: 28 set. 2026.

EXPO. **Expo documentation**. [S. l.], [2026a]. Disponível em: <https://docs.expo.dev/>. Acesso em: 28 set. 2026.

EXPO. **Introduction to Expo Router**. [S. l.], [2026b]. Disponível em: <https://docs.expo.dev/router/introduction/>. Acesso em: 28 set. 2026.

FASTIFY. **Fastify documentation**. [S. l.], [2026]. Disponível em:
<https://fastify.dev/docs/latest/>. Acesso em: 28 set. 2026.

FETTE, Ian; MELNIKOV, Alexey. **RFC 6455**: the WebSocket protocol. [S. l.]: IETF, 2011. Disponível em: <https://www.rfc-editor.org/rfc/rfc6455>. Acesso em: 28 set. 2026.

FIELDING, Roy; NOTTINGHAM, Mark; RESCHKE, Julian (ed.). **RFC 9110**: HTTP semantics. [S. l.]: IETF, 2022. Disponível em: <https://www.rfc-editor.org/rfc/rfc9110>. Acesso em: 28 set. 2026.

FOWLER, Martin. **CQRS**. [S. l.], 14 jul. 2011. Disponível em:
<https://martinfowler.com/bliki/CQRS.html>. Acesso em: 28 set. 2026.

FOWLER, Martin. **CircuitBreaker**. [S. l.], 6 mar. 2014. Disponível em:
<https://martinfowler.com/bliki/CircuitBreaker.html>. Acesso em: 28 set. 2026.

GOODCHILD, Michael F. Citizens as sensors: the world of volunteered geography. **GeoJournal**, [s. l.], v. 69, n. 4, p. 211-221, 2007. DOI:
<https://doi.org/10.1007/s10708-007-9111-y>.

JONES, Michael; BRADLEY, John; SAKIMURA, Nat. **RFC 7519**: JSON Web Token (JWT). [S. l.]: IETF, 2015. Disponível em: <https://www.rfc-editor.org/rfc/rfc7519>. Acesso em: 28 set. 2026.

KLEPPMANN, Martin. **Designing data-intensive applications**: the big ideas behind reliable, scalable, and maintainable systems. Sebastopol: O'Reilly Media, 2017.

NEWMAN, Sam. **Criando microsserviços**: projetando sistemas com componentes menores e mais especializados. 2. ed. São Paulo: Novatec, 2022.

NGINX. **Module ngx_http_upstream_module**. [S. l.], [2026a]. Disponível em:
https://nginx.org/en/docs/http/ngx_http_upstream_module.html. Acesso em: 28 set. 2026.

NGINX. **WebSocket proxying**. [S. l.], [2026b]. Disponível em:
<https://nginx.org/en/docs/http/websocket.html>. Acesso em: 28 set. 2026.

NYGARD, Michael T. **Release it!**: design and deploy production-ready software. 2. ed. Raleigh: Pragmatic Bookshelf, 2018.

OASIS. **MQTT version 3.1.1**. Edited by Andrew Banks and Rahul Gupta. [S. l.]: OASIS, 29 out. 2014. OASIS Standard. Disponível em:

<https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>. Acesso em: 28 set. 2026.

OPEN-METEO. **Air quality API**. [S. l.], [2026a]. Disponível em: <https://open-meteo.com/en/docs/air-quality-api>. Acesso em: 28 set. 2026.

OPEN-METEO. **Weather forecast API**. [S. l.], [2026b]. Disponível em: <https://open-meteo.com/en/docs>. Acesso em: 28 set. 2026.

OPENSTREETMAP. **Copyright and license**. [S. l.], [2026]. Disponível em: <https://www.openstreetmap.org/copyright>. Acesso em: 28 set. 2026.

OPOSSUM. **Opossum documentation**. [S. l.], [2026]. Disponível em: <https://nodeshift.dev/opossum/>. Acesso em: 28 set. 2026.

PLAYWRIGHT. **Playwright documentation**. [S. l.], [2026]. Disponível em: <https://playwright.dev/docs/intro>. Acesso em: 28 set. 2026.

POSTGIS. **PostGIS 3.6 manual**. [S. l.], [2026]. Disponível em: <https://postgis.net/docs/manual-3.6/>. Acesso em: 28 set. 2026.

POSTGRESQL. **PostgreSQL 18 documentation**. [S. l.]: The PostgreSQL Global Development Group, [2026]. Disponível em: <https://www.postgresql.org/docs/18/>. Acesso em: 28 set. 2026.

RABBITMQ. **AMQP 0-9-1 model explained**. [S. l.], [2026a]. Disponível em: <https://www.rabbitmq.com/tutorials/amqp-concepts>. Acesso em: 28 set. 2026.

RABBITMQ. **MQTT plugin**. [S. l.], [2026b]. Disponível em: <https://www.rabbitmq.com/docs/mqtt>. Acesso em: 28 set. 2026.

RABBITMQ. **Consumer acknowledgements and publisher confirms**. [S. l.], [2026c]. Disponível em: <https://www.rabbitmq.com/docs/confirmations>. Acesso em: 28 set. 2026.

RABBITMQ. **Dead letter exchanges**. [S. l.], [2026d]. Disponível em: <https://www.rabbitmq.com/docs/dlx>. Acesso em: 28 set. 2026.

REACT NATIVE. **Introduction**. [S. l.], [2026a]. Disponível em: <https://reactnative.dev/docs/getting-started>. Acesso em: 28 set. 2026.

REACT NATIVE. **Platform-specific code**. [S. l.], [2026b]. Disponível em: <https://reactnative.dev/docs/platform-specific-code>. Acesso em: 28 set. 2026.

RICHARDSON, Chris. **Microservices patterns**: with examples in Java. Shelter Island: Manning, 2018.

RICHARDSON, Chris. **Pattern**: transactional outbox. [S. l.], [2026a]. Disponível em: <https://microservices.io/patterns/data/transactional-outbox.html>. Acesso em: 28 set. 2026.

RICHARDSON, Chris. **Pattern**: API gateway / backends for frontends. [S. l.], [2026b]. Disponível em: <https://microservices.io/patterns/apigateway.html>. Acesso em: 28 set. 2026.

SÃO PAULO (Estado). Lei nº 12.233, de 16 de janeiro de 2006. Define a Área de Proteção e Recuperação dos Mananciais da Bacia Hidrográfica do Guarapiranga. São Paulo: Assembleia Legislativa, 2006. Disponível em: <https://www.al.sp.gov.br/repositorio/legislacao/lei/2006/lei-12233-16.01.2006.html>. Acesso em: 28 set. 2026.

SÃO PAULO (Estado). Lei nº 13.579, de 13 de julho de 2009. Define a Área de Proteção e Recuperação dos Mananciais da Bacia Hidrográfica do Reservatório Billings - APRM-B. São Paulo: Assembleia Legislativa, 2009. Disponível em: <https://www.al.sp.gov.br/repositorio/legislacao/lei/2009/lei-13579-13.07.2009.html>. Acesso em: 28 set. 2026.

SOCKET.IO. **Socket.IO documentation**: introduction. [S. l.], [2026]. Disponível em: <https://socket.io/docs/v4/>. Acesso em: 28 set. 2026.

TANENBAUM, Andrew S.; VAN STEEN, Maarten. **Sistemas distribuídos**: princípios e paradigmas. 2. ed. São Paulo: Pearson Prentice Hall, 2007.

VITEST. **Getting started**. [S. l.], [2026]. Disponível em: <https://vitest.dev/guide/>. Acesso em: 28 set. 2026.

10 FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS